

El síndrome metabólico constituye un conjunto de alteraciones clínicas caracterizadas por la coexistencia de factores de riesgo como hipertensión arterial, dislipemia y diabetes mellitus tipo 2, cuya presencia simultánea incrementa de forma significativa el riesgo cardiovascular. Diversos estudios han demostrado que los pacientes con síndrome metabólico presentan una mayor probabilidad de sufrir eventos adversos como infarto agudo de miocardio, ictus y mortalidad cardiovascular, debido a la interacción sinérgica de estos factores sobre el sistema vascular (World Health Organization – Cardiovascular diseases [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)]).

En este contexto, la progresión del síndrome metabólico suele ir acompañada de una intensificación progresiva del tratamiento farmacológico, reflejando un empeoramiento del control clínico y un aumento del riesgo global del paciente. Sin embargo, la identificación temprana de aquellos pacientes con mayor probabilidad de intensificación terapéutica sigue siendo un reto en la práctica clínica, especialmente en entornos con información limitada.

El presente estudio tiene como objetivo analizar patrones de evolución terapéutica en pacientes con patologías asociadas al síndrome metabólico y desarrollar un modelo predictivo capaz de estimar la probabilidad de intensificación del tratamiento a partir del perfil inicial del paciente. Este enfoque permite avanzar hacia estrategias de estratificación de riesgo más proactivas, orientadas a optimizar el seguimiento clínico y prevenir la progresión hacia eventos cardiovasculares mayores.

En este estudio se adopta un enfoque basado en datos de dispensación farmacéutica, utilizando el consumo de medicamentos como proxy de las condiciones clínicas del paciente. Aunque no se dispone de parámetros analíticos ni medidas antropométricas, el patrón de tratamiento permite inferir de forma indirecta la presencia y evolución de patologías asociadas al síndrome metabólico. Este planteamiento abre la puerta a un seguimiento más proactivo y accesible en entornos de práctica real, donde la información clínica completa no siempre está disponible, si bien sus resultados deben interpretarse como una aproximación complementaria y no sustitutiva de la evaluación clínica directa. Uno de los objetivos de este trabajo es formar evidencia para sostener el seguimiento clínico-terapéutico del paciente desde la oficina de farmacia y así facilitar la adherencia al tratamiento y la comunicación con los miembros implicados en todo el proceso.

SELECCIÓN DE VARIABLES A ANALIZAR

La selección de datos a analizar en este trabajo se basa en los siguientes protocolos de consenso internacional sobre los principales tratamientos de las patologías que conforman el síndrome metabólico:

1. NCEP ATP III (National Cholesterol Education Program), que define que el síndrome metabólico se da si se cumplen al menos 3 de las siguientes condiciones: presión arterial elevada (HTA), glucosa elevada o diabetes, triglicéridos elevados, HDL bajo y/o obesidad abdominal (Expert Panel on Detection, Evaluation, and Treatment of High Blood Cholesterol in Adults (2001)).
2. IDF (International Diabetes Federation), que aporta una definición global del concepto, hace especial énfasis en la obesidad y afirma que los principales factores a tener en cuenta son HTA, glucosa y lípidos (Alberti KGMM et al. (2005). IDF Consensus Definition of Metabolic Syndrome.)).
3. AHA / NHLBI (American Heart Association / National Heart, Lung, and Blood Institute), que actualiza lo visto en ATP III y aún así mantiene los mismos parámetros a medir (Grundy SM et al. (2005). Diagnosis and Management of the Metabolic Syndrome. Circulation.).

Basándose en la evidencia, se decide utilizar los datos de dispensación de los siguientes principios activos: enalapril, enalapril/hidroclorotiazida y amlodipino para HTA, simvastatina para colesterol y metformina para azúcar. Dado que el fin principal de este trabajo es realiar una demostración de habilidad de lo aprendido en el máster de Big Data y Data Science, también se debe aclarar que: se eligen los medicamentos genéricos de la marca Cinfa por ser los más utilizados en esta farmacia, no se utilizan datos de especialidad y tampoco se tienen en cuenta otros medicamentos, como pueden ser lisinopril o lercanidipino para HTA, rosuvastatina para colesterol o sitagliptina para azúcar porque los principios activos ya elegidos conforman el mayor grueso de ventas de cada grupo en la farmacia. Con el fin de no extender demasiado el proyecto, y entendiendo que son las mejores reresentantes de cada grupo, se opta por estas variables.

Los datos utilizados en este estudio proceden de registros de dispensación farmacéutica correspondientes al año 2025, obtenidos a partir de una farmacia comunitaria ubicada en la Comunidad de Madrid. Estos registros incluyen información anonimizada sobre las dispensaciones realizadas, permitiendo reconstruir la evolución terapéutica de los pacientes a lo largo del periodo analizado.

El conjunto de datos recoge variables relacionadas con el tipo de medicamento dispensado, la fecha de la dispensación y un identificador de paciente anonimizado, lo que posibilita el seguimiento longitudinal sin comprometer la privacidad. A partir de esta información, se han identificado patrones de tratamiento asociados a patologías vinculadas al síndrome metabólico, como hipertensión, dislipemia y diabetes.

Es importante destacar que los datos reflejan la práctica real en un entorno de farmacia comunitaria, lo que aporta un enfoque de “real world data” (RWD), pero también implica ciertas limitaciones, como la ausencia de información clínica directa (por ejemplo, valores analíticos o diagnósticos médicos), lo que condiciona la interpretación de los resultados.

BLOQUE 1 — Carga de librerías y definición de rutas

```
In [6]: # pandas → para manipulación de datos
import pandas as pd

# numpy → para operaciones numéricas
import numpy as np

# matplotlib → visualización básica
import matplotlib.pyplot as plt

# seaborn → visualización más avanzada
import seaborn as sns

# configuración estética de gráficos
sns.set(style="whitegrid")

# Ruta donde están todos los archivos
ruta_base = r"C:\Users\USUARIO\Ejercicios Master\Sindrome metabolico"

# Diccionario de archivos

# Creamos un diccionario para tener todos los archivos organizados

# Clave → nombre lógico

# Valor → nombre del archivo (sin extensión todavía)

archivos = {
    "amlodipino_5": "Amlodipino5",
    "amlodipino_10": "Amlodipino10",

    "enalapril_5": "Enalapril5",
    "enalapril_10": "Enalapril10",
    "enalapril_20": "Enalapril20",
    "enalapril_hidro": "EnalaprilHidro",

    "metformina_cinfa": "MeforminaCinfa",
    "metformina_uxa": "MetforminaUXA",

    "simva_10": "Simva10",
    "simva_20": "Simva20",
    "simva_40": "Simva40"
}

# Función para construir la ruta completa (ahora .html)
def obtener_ruta(nombre_archivo):
    return f"{ruta_base}\\{nombre_archivo}.html"
# (De momento no cargamos nada, solo dejamos todo preparado)
```

```
In [7]: import os
from bs4 import BeautifulSoup
import pandas as pd
import re

# Lista donde vamos a acumular TODOS Los registros de TODOS Los archivos
data_total = []

# Recorremos todos Los archivos de La carpeta
for file in os.listdir(ruta_base):

    # Nos aseguramos de procesar solo archivos HTML (evita errores con otros formatos)
    if file.lower().endswith((".htm", ".html")):

        print("Procesando:", file) # Para saber qué archivo estamos Leyendo

        # Construimos La ruta completa del archivo
        ruta = os.path.join(ruta_base, file)

        # Abrimos el archivo en modo binario (esto evita problemas de codificación)
        with open(ruta, "rb") as f:

            # Parseamos el HTML con BeautifulSoup
            soup = BeautifulSoup(f.read(), "html.parser")

            # Extraemos TODO el texto del HTML (quitamos etiquetas)
            text = soup.get_text()

            # Dividimos el texto en líneas
            lines = text.split("\n")

            # Limpiamos líneas:
            # - quitamos espacios
            # - eliminamos líneas vacías
            lines = [line.strip() for line in lines if line.strip()]

            # FILTRADO DE LÍNEAS RELEVANTES

            # Aquí nos quedamos SOLO con Las líneas que contienen datos reales
            # Es decir:
            # - Líneas que empiezan por fecha (ej: 02/01/25)
            # - o líneas que empiezan por "---" (continuación sin fecha)
            data_lines = []

            for line in lines:
                if re.match(r"(\d{2}/\d{2}/\d{2}|--)", line):
                    data_lines.append(line)

            # PARSING DE LAS LÍNEAS (CONVERSIÓN A COLUMNAS)

            registros = [] # Lista de filas del archivo actual
            fecha_actual = None # aquí guardamos La última fecha válida

            for line in data_lines:

                # Dividimos La línea en "trozos" (columnas separadas por espacios)
                partes = line.split()

                # CASO 1: LÍNEA CON FECHA (ej: 02/01/25 ...)
                if re.match(r"\d{2}/\d{2}/\d{2}", partes[0]):

                    # Guardamos La fecha → será referencia para Las siguientes líneas "---"
                    fecha_actual = partes[0]

                    # Nos aseguramos de que La línea tiene suficientes columnas
                    if len(partes) >= 11:

                        # Construimos el registro (fila)
                        row = {
                            "fecha": partes[0], # fecha de La dispensación
                            "hora": partes[1], # hora

                            # "A crédito" viene separado en dos columnas → Lo unimos
                            "operacion": partes[2] + (" " + partes[3] if partes[2] == "A" else ""),

                            # Ajustamos posiciones según si hay "A crédito"
                            "cliente": partes[3] if partes[2] != "A" else partes[4],
                            "vendedor": partes[4] if partes[2] != "A" else partes[5],
                            "org": partes[5] if partes[2] != "A" else partes[6],
                            "pvp": partes[6] if partes[2] != "A" else partes[7],
                            "cantidad": partes[7] if partes[2] != "A" else partes[8],
                            "imp_bruto": partes[8] if partes[2] != "A" else partes[9],
                            "dto": partes[9] if partes[2] != "A" else partes[10],
                            "imp_neto": partes[10] if partes[2] != "A" else partes[11],

                            # MUY IMPORTANTE → trazabilidad del origen
                            "archivo": file
                        }

                        registros.append(row)

                # CASO 2: LÍNEA SIN FECHA ("---")
                elif partes[0] == "---" and fecha_actual is not None:

                    # Usamos La última fecha conocida (esto es CLAVE)
                    if len(partes) >= 11:

                        row = {
                            "fecha": fecha_actual, # heredamos La fecha anterior
                            "hora": partes[1],
                            "operacion": partes[2] + (" " + partes[3] if partes[2] == "A" else ""),
                            "cliente": partes[3] if partes[2] != "A" else partes[4],
                            "vendedor": partes[4] if partes[2] != "A" else partes[5],
                            "org": partes[5] if partes[2] != "A" else partes[6],
                            "pvp": partes[6] if partes[2] != "A" else partes[7],
                            "cantidad": partes[7] if partes[2] != "A" else partes[8],
                            "imp_bruto": partes[8] if partes[2] != "A" else partes[9],
                            "dto": partes[9] if partes[2] != "A" else partes[10],
                            "imp_neto": partes[10] if partes[2] != "A" else partes[11],
                            "archivo": file
                        }

                        registros.append(row)

            # Añadimos todos Los registros de este archivo al dataset global
```

```
data_total.extend(registros)

# Convertimos todo a DataFrame final
df = pd.DataFrame(data_total)

# Vista rápida
df.head()
```

Procesando: Amlodipino10.htm
Procesando: Amlodipino5.htm
Procesando: Enalapril10.htm
Procesando: Enalapril20.htm
Procesando: Enalapril5.htm
Procesando: EnalaprilHidro.htm
Procesando: MetforminaCinfa.htm
Procesando: MetforminaUXA.htm
Procesando: Simva10.htm
Procesando: Simva20.htm
Procesando: Simva40.htm

Out[7]:

	fecha	hora	operacion	cliente	vendedor	org	pvp	cantidad	imp_bruto	dto	imp_net	archivo
0	02/01/25	16:38	Contado	484	3	161	2,50	1	0,00	0,00	0,00	Amlodipino10.htm
1	07/01/25	13:01	Contado	487	9	162	2,50	1	0,25	0,00	0,25	Amlodipino10.htm
2	07/01/25	13:18	Contado	7572	5	161	2,50	1	0,00	0,00	0,00	Amlodipino10.htm
3	10/01/25	16:30	Contado	8136	3	001	2,50	1	2,50	0,00	2,50	Amlodipino10.htm
4	11/01/25	13:09	Contado	293	9	162	2,50	1	0,25	0,00	0,25	Amlodipino10.htm

La construcción del dataset a partir de múltiples archivos de dispensación permite integrar información de diferentes medicamentos en una única estructura, facilitando el análisis longitudinal por paciente. La inspección inicial evidencia la presencia de variables administrativas y económicas (como `pvp`, `imp_bruto` o `dto`) junto a variables operativas (`operacion`, `hora`), que, aunque relevantes en el contexto de la farmacia, no aportan valor directo al objetivo clínico del estudio. Asimismo, se observan valores faltantes y registros asociados a distintas tipologías de operación (venta, administración, recepción), lo que refleja la naturaleza no estructurada de los datos de práctica real. Este análisis inicial justifica la necesidad de un proceso de depuración y selección de variables enfocado en aislar la información verdaderamente útil para reconstruir la evolución terapéutica de los pacientes.

BLOQUE 3 — Limpieza y estandarización básica

```
In [8]: # Copiamos el dataframe original
df_clean = df.copy()

# 1. LIMPIEZA DE CLIENTE (ID PACIENTE)

# Convertimos a numérico
df_clean["cliente"] = pd.to_numeric(df_clean["cliente"], errors="coerce")

# Eliminamos registros sin cliente (no sirven para análisis por paciente)
df_clean = df_clean.dropna(subset=["cliente"])

# 2. LIMPIEZA DE FECHA

# Convertimos a datetime (formato dd/mm/yy)
df_clean["fecha"] = pd.to_datetime(
    df_clean["fecha"],
    format="%d/%m/%y",
    errors="coerce"
)

# Eliminamos fechas inválidas
df_clean = df_clean.dropna(subset=["fecha"])

# 3. LIMPIEZA DE VARIABLES NUMÉRICAS

columnas_numericas = ["pvp", "cantidad", "imp_bruto", "dto", "imp_net"]

for col in columnas_numericas:

    # Pasamos todo a string
    df_clean[col] = df_clean[col].astype(str)

    # Cambiamos coma decimal por punto
    df_clean[col] = df_clean[col].str.replace(",", ".")

    # Convertimos a float
    df_clean[col] = pd.to_numeric(df_clean[col], errors="coerce")

# 4. RESET INDEX (importante tras eliminar filas)
df_clean = df_clean.reset_index(drop=True)

# 5. CHECK FINAL

print("Shape final:", df_clean.shape)
df_clean.head()
```

Shape final: (4793, 12)

Out[8]:

	fecha	hora	operacion	cliente	vendedor	org	pvp	cantidad	imp_bruto	dto	imp_net	archivo
0	2025-01-02	16:38	Contado	484.0	3	161	2.5	1.0	0.00	0.0	0.00	Amlodipino10.htm
1	2025-01-07	13:01	Contado	487.0	9	162	2.5	1.0	0.25	0.0	0.25	Amlodipino10.htm
2	2025-01-07	13:18	Contado	7572.0	5	161	2.5	1.0	0.00	0.0	0.00	Amlodipino10.htm
3	2025-01-10	16:30	Contado	8136.0	3	001	2.5	1.0	2.50	0.0	2.50	Amlodipino10.htm
4	2025-01-11	13:09	Contado	293.0	9	162	2.5	1.0	0.25	0.0	0.25	Amlodipino10.htm

El proceso de limpieza y estandarización permite transformar un conjunto de datos heterogéneo en una estructura coherente y analizable, corrigiendo inconsistencias de formato (como separadores decimales), gestionando valores no válidos y asegurando la correcta tipificación de las variables. La conversión de los datos a formatos numéricos y la reorganización del índice facilitan su uso en etapas posteriores de análisis y modelado. Este paso resulta fundamental en datos de práctica real, donde la falta de estandarización inicial puede introducir errores si no se aborda adecuadamente, garantizando así la fiabilidad de los resultados obtenidos.

Gracias a este proceso, ya setienen identificados a los pacientes con ficha propia en la farmacia, eliminando el resto de dispensaciones de pacientes no registrados a los que no se les puede hacer el seguimiento.

BLOQUE 4 — Identificación de medicamento, dosis y grupo terapéutico


```
In [9]: # Copiamos el dataset limpio
df_med = df_clean.copy()

# 1. IDENTIFICAR MEDICAMENTO A PARTIR DEL ARCHIVO

def identificar_medicamento(nombre_archivo):

    nombre = nombre_archivo.lower()

    if "amlodipino" in nombre:
        return "amlodipino"

    elif "enalaprilhidro" in nombre:
        return "enalapril_hidro"

    elif "enalapril" in nombre:
        return "enalapril"

    elif "metformina" in nombre:
        return "metformina"

    elif "simva" in nombre:
        return "simvastatina"

    else:
        return "otro"

df_med["medicamento"] = df_med["archivo"].apply(identificar_medicamento)

# 2. IDENTIFICAR DOSIS A PARTIR DEL NOMBRE DEL ARCHIVO

def identificar_dosis(nombre_archivo):

    nombre = nombre_archivo.lower()

    # Amlodipino
    if "amlodipino5" in nombre:
        return 5
    elif "amlodipino10" in nombre:
        return 10

    # Enalapril
    elif "enalapril5" in nombre:
        return 5
    elif "enalapril10" in nombre:
        return 10
    elif "enalapril20" in nombre:
        return 20
    elif "enalaprilhidro" in nombre:
        return 20 # Lo tratamos como nivel más alto

    # Simvastatina
    elif "simva10" in nombre:
        return 10
    elif "simva20" in nombre:
        return 20
    elif "simva40" in nombre:
        return 40

    # Metformina
    elif "metformina" in nombre:
        return 1 # placeholder

    else:
        return None

df_med["dosis"] = df_med["archivo"].apply(identificar_dosis)

# 3. AGRUPAR POR GRUPO TERAPÉUTICO

def identificar_grupo(medicamento):

    if medicamento in ["amlodipino", "enalapril", "enalapril_hidro"]:
        return "hta"

    elif medicamento == "simvastatina":
        return "colesterol"

    elif medicamento == "metformina":
        return "diabetes"

    else:
        return "otro"

df_med["grupo"] = df_med["medicamento"].apply(identificar_grupo)

# 4. CHECK RÁPIDO

print(df_med[["archivo", "medicamento", "dosis", "grupo"]].drop_duplicates().sort_values(by="archivo"))
```

	archivo	medicamento	dosis	grupo
0	Amlodipino10.htm	amlodipino	10	hta
310	Amlodipino5.htm	amlodipino	5	hta
1051	Enalapril10.htm	enalapril	10	hta
1120	Enalapril20.htm	enalapril	20	hta
1484	Enalapril5.htm	enalapril	5	hta
1641	EnalaprilHidro.htm	enalapril_hidro	20	hta
2279	MetforminaCinfa.htm	metformina	1	diabetes
2962	MetforminaUXA.htm	metformina	1	diabetes
3080	Simva10.htm	simvastatina	10	colesterol
3474	Simva20.htm	simvastatina	20	colesterol
4606	Simva40.htm	simvastatina	40	colesterol

La clasificación de los medicamentos en grupos terapéuticos (hipertensión, colesterol y diabetes) permite transformar los datos de dispensación en información clínicamente interpretable, facilitando el análisis de la carga metabólica de cada paciente. A partir de la identificación del fármaco y su dosis, se establece una correspondencia directa entre tratamiento farmacológico y patología subyacente, lo que resulta clave en ausencia de variables clínicas explícitas.

Esta categorización simplifica la complejidad del dataset original y constituye la base para reconstruir la evolución terapéutica y analizar patrones de intensificación, alineando el análisis con los principales componentes del síndrome metabólico.


```
In [10]: # Copiamos dataset
df_evo = df_med.copy()

# 1. ORDENAMOS LOS DATOS

# Orden cronológico por paciente
df_evo = df_evo.sort_values(by=["cliente", "fecha"])

# 2. IDENTIFICAR CAMBIOS DE DOSIS

# Comparamos cada fila con la anterior del mismo paciente y mismo medicamento
df_evo["dosis_prev"] = df_evo.groupby(["cliente", "medicamento"])["dosis"].shift(1)

# Detectamos si hay cambio
df_evo["cambio_dosis"] = df_evo["dosis"] != df_evo["dosis_prev"]

# 3. IDENTIFICAR ESCALADO (SUBIDA DE DOSIS)

df_evo["escalado"] = df_evo["dosis"] > df_evo["dosis_prev"]

# 4. IDENTIFICAR NUEVO TRATAMIENTO

# Primera aparición de un medicamento en ese paciente
df_evo["nuevo_tratamiento"] = df_evo.groupby(["cliente", "medicamento"]).cumcount() == 0

# 5. VISUALIZACIÓN DE EJEMPLO (UN PACIENTE)

paciente_ejemplo = df_evo["cliente"].iloc[0]

df_evo[df_evo["cliente"] == paciente_ejemplo][
    ["fecha", "medicamento", "dosis", "cambio_dosis", "escalado", "nuevo_tratamiento"]
]
```

Out[10]:

	fecha	medicamento	dosis	cambio_dosis	escalado	nuevo_tratamiento
1309	2025-01-02	enalapril	20	True	False	True
1964	2025-01-04	enalapril_hidro	20	True	False	True
1965	2025-01-04	enalapril_hidro	20	False	False	False
1966	2025-01-04	enalapril_hidro	20	False	False	False
1967	2025-01-04	enalapril_hidro	20	False	False	False
...	...	...	...	...	...	...
4593	2025-12-22	simvastatina	20	True	True	False
4594	2025-12-22	simvastatina	20	False	False	False
2274	2025-12-26	enalapril_hidro	20	False	False	False
3079	2025-12-30	metformina	1	False	False	False
4603	2025-12-30	simvastatina	20	False	False	False

226 rows × 6 columns

El análisis de la evolución terapéutica a nivel individual permite reconstruir la trayectoria de tratamiento de cada paciente, identificando cambios de dosis, incorporación de nuevos fármacos y posibles escalados terapéuticos. Este enfoque pone de manifiesto cómo los datos de dispensación pueden utilizarse para inferir dinámicas clínicas complejas, incluso en ausencia de información médica directa.

Más allá del caso individual analizado, el conjunto de datos muestra patrones consistentes en la evolución de los pacientes, caracterizados por una progresión gradual desde tratamientos monoterapia hacia combinaciones más complejas, especialmente en presencia de múltiples factores metabólicos. Asimismo, se observa que muchos pacientes mantienen largos periodos de estabilidad terapéutica, interrumpidos por cambios puntuales que reflejan posibles ajustes clínicos.

En conjunto, este bloque evidencia que es posible modelar la evolución del tratamiento a partir de datos de dispensación, sentando las bases para identificar patrones de progresión y futuros procesos de intensificación terapéutica.

El paciente 484 presenta una evolución terapéutica compatible con la progresión de un síndrome metabólico a lo largo de 2025. El 02/01/2025 inicia registro con enalapril a dosis de 20 mg, lo que sugiere un contexto previo de hipertensión arterial ya establecida o mal controlada. Tan solo dos días después, el 04/01/2025, se observa una intensificación del tratamiento con la introducción de enalapril combinado con hidroclorotiazida (equivalente a un nivel terapéutico superior), manteniéndose posteriormente esta pauta sin cambios aparentes durante gran parte del año, lo que indica una fase de estabilidad clínica en el control tensional.

Hacia finales del periodo, el 22/12/2025, se introduce simvastatina a dosis de 20 mg, con evidencia de escalado, lo que sugiere aparición o empeoramiento de dislipemia. Finalmente, el 30/12/2025 se incorpora metformina (sin especificación de dosis en los datos disponibles), coexistiendo con simvastatina 20 mg y enalapril con hidroclorotiazida, configurando un escenario de triple terapia característico de un síndrome metabólico plenamente establecido.

Este patrón temporal refleja una progresión desde hipertensión arterial aislada hacia una afectación metabólica múltiple, si bien debe considerarse como limitación que los datos proceden de dispensaciones y no permiten inferir directamente la adherencia ni la prescripción clínica exacta.

5.1 Metformina (continuidad Cinfa + UXA)
En 2025 se descontinuo por problemas de abastecimiento la metformina de cinfa, se sustituyó por la de UXA. Se deben unir ambos archivos para mantener una continuidad terapéutica real y consistencia del dato.

```
In [11]: # Copiamos dataset
df_int = df_med.copy()

# 1. UNIFICAR METFORMINA (CINFA + UXA)

# Creamos una variable más específica de origen
def tipo_metformina(nombre_archivo):

    nombre = nombre_archivo.lower()

    if "cinfa" in nombre:
        return "cinfa"

    elif "uxa" in nombre:
        return "uxa"

    else:
        return None

df_int["origen_metformina"] = df_int["archivo"].apply(tipo_metformina)
```

```
# 2. FORZAMOS QUE TODA METFORMINA SEA UNA SOLA ENTIDAD

df_int.loc[df_int["medicamento"] == "metformina", "medicamento"] = "metformina"

# (ya lo era, pero dejamos claro que es intencional)

# 3. ORDENAMOS PARA VER CONTINUIDAD

df_int = df_int.sort_values(by=["cliente", "fecha"])
```

```
In [12]: df_int[df_int["medicamento"] == "metformina"][
         ["cliente", "fecha", "archivo", "origen_metformina"]
         ].head(20)
```

Out[12]:

	cliente	fecha	archivo	origen_metformina
2654	1.0	2025-01-17	MetforminaCinfa.htm	cinfa
2672	1.0	2025-01-30	MetforminaCinfa.htm	cinfa
2673	1.0	2025-01-31	MetforminaCinfa.htm	cinfa
2694	1.0	2025-02-11	MetforminaCinfa.htm	cinfa
2730	1.0	2025-03-10	MetforminaCinfa.htm	cinfa
2731	1.0	2025-03-10	MetforminaCinfa.htm	cinfa
2745	1.0	2025-03-19	MetforminaCinfa.htm	cinfa
2756	1.0	2025-03-24	MetforminaCinfa.htm	cinfa
2780	1.0	2025-04-10	MetforminaCinfa.htm	cinfa
2795	1.0	2025-04-30	MetforminaCinfa.htm	cinfa
2812	1.0	2025-05-10	MetforminaCinfa.htm	cinfa
2813	1.0	2025-05-12	MetforminaCinfa.htm	cinfa
2815	1.0	2025-05-12	MetforminaCinfa.htm	cinfa
2829	1.0	2025-05-23	MetforminaCinfa.htm	cinfa
2837	1.0	2025-05-29	MetforminaCinfa.htm	cinfa
2850	1.0	2025-06-19	MetforminaCinfa.htm	cinfa
2851	1.0	2025-06-19	MetforminaCinfa.htm	cinfa
2856	1.0	2025-06-23	MetforminaCinfa.htm	cinfa
2857	1.0	2025-06-23	MetforminaCinfa.htm	cinfa
2865	1.0	2025-06-30	MetforminaCinfa.htm	cinfa

5.2 Bloque hipertensión

Tanto enalapril como amlodipino son tratamientos de HTA. Enalapril, en esta farmacia, es el antihipertensivo más vendido. Sin embargo, el cambio de enalapril a amlodipino se interpreta como una posible sustitución terapéutica, potencialmente asociada a efectos adversos o intolerancia, más que como una intensificación del tratamiento antihipertensivo. Se estudiará la escalada intra-fármaco y el swtich entre fármacos.

```
In [13]: # bloque previo obligatorio – creación de df_hta_clean

df_hta_clean = df_med.copy()

# filtrar solo hta
df_hta_clean = df_hta_clean[df_hta_clean["grupo"] == "hta"].copy()

# solo dispensaciones reales
df_hta_clean = df_hta_clean[
    df_hta_clean["operacion"].isin(["Contado", "A crédito"])
]

# ordenar
df_hta_clean = df_hta_clean.sort_values(by=["cliente", "fecha"])
```

```
In [14]: # crear una variable de intensidad terapéutica hta

def nivel_hta(row):

    med = row["medicamento"]
    dosis = row["dosis"]

    # amlodipino
    if med == "amlodipino":
        if dosis == 5:
            return 1
        elif dosis == 10:
            return 2

    # enalapril
    elif med == "enalapril":
        if dosis == 5:
            return 2
        elif dosis == 10:
            return 3
        elif dosis == 20:
            return 4

    # enalapril + hidro (nivel más alto)
    elif med == "enalapril_hidro":
        return 5

    return None

df_hta_clean["nivel_hta"] = df_hta_clean.apply(nivel_hta, axis=1)

# ordenar bien
df_hta_clean = df_hta_clean.sort_values(by=["cliente", "fecha"])

# nivel previo
df_hta_clean["nivel_prev"] = df_hta_clean.groupby("cliente")["nivel_hta"].shift(1)

# escalado real
df_hta_clean["escalado_real"] = df_hta_clean["nivel_hta"] > df_hta_clean["nivel_prev"]

# detección de cambios
df_hta_clean["cambio_farmaco"] = (
    df_hta_clean["medicamento"] != df_hta_clean.groupby("cliente")["medicamento"].shift(1)
)
```

```
# identificacion de pacientes
escalado_hta_real = df_hta_clean.groupby("cliente")["escalado_real"].any().reset_index()

print(escalado_hta_real["escalado_real"].value_counts())
```

escalado_real
False 166
True 18
Name: count, dtype: int64

In [15]: # 1. quedarnos con los pacientes que escalan

```
pacientes_hta_escalado_real = df_hta_clean[
    df_hta_clean["escalado_real"] == True
][["cliente"]].unique()

print(pacientes_hta_escalado_real)
```

[202. 373. 469. 771. 892. 1643. 2033. 2269. 2272. 2734. 2916. 2991.
 4159. 4713. 5405. 7607. 8570. 8951.]

In [16]: # 2. ver evolución completa

```
df_hta_clean[
    df_hta_clean["cliente"].isin(pacientes_hta_escalado_real)
].sort_values(by=["cliente", "fecha"])
```

Out[16]:

	fecha	hora	operacion	cliente	vendedor	org	pvp	cantidad	imp_bruto	dto	imp_net	archivo	medicamento	dosis	grupo	nivel_hta	nivel_prev	escalado_real	car
359	2025-02-15	21:10	Contado	202.0	3	001	1.25	1.0	1.25	0.0	1.25	Amlodipino5.htm	amlodipino	5	hta	1	NaN	False	
1842	2025-08-28	09:16	A crédito	202.0	6	162	1.84	1.0	0.00	0.0	0.00	EnalaprilHidro.htm	enalapril_hidro	20	hta	5	1.0	True	
310	2025-01-02	11:44	A crédito	373.0	5	162	1.25	1.0	0.13	0.0	0.13	Amlodipino5.htm	amlodipino	5	hta	1	NaN	False	
311	2025-01-02	11:44	A crédito	373.0	5	162	1.25	1.0	0.13	0.0	0.13	Amlodipino5.htm	amlodipino	5	hta	1	1.0	False	
1641	2025-01-02	11:44	A crédito	373.0	5	162	1.84	1.0	0.18	0.0	0.18	EnalaprilHidro.htm	enalapril_hidro	20	hta	5	1.0	True	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1795	2025-07-09	18:07	Contado	8951.0	3	164	1.84	1.0	0.92	0.0	0.92	EnalaprilHidro.htm	enalapril_hidro	20	hta	5	5.0	False	
1853	2025-09-13	10:14	Contado	8951.0	9	164	1.84	1.0	0.92	0.0	0.92	EnalaprilHidro.htm	enalapril_hidro	20	hta	5	5.0	False	
552	2025-10-15	18:54	Contado	8951.0	1	164	1.25	1.0	0.13	0.0	0.13	Amlodipino5.htm	amlodipino	5	hta	1	5.0	False	
1878	2025-10-15	18:54	Contado	8951.0	1	164	1.84	1.0	0.92	0.0	0.92	EnalaprilHidro.htm	enalapril_hidro	20	hta	5	1.0	True	
1879	2025-10-15	18:54	Contado	8951.0	1	164	1.84	1.0	0.92	0.0	0.92	EnalaprilHidro.htm	enalapril_hidro	20	hta	5	5.0	False	

178 rows × 19 columns

En los pacientes con escalado real de tratamiento antihipertensivo se observan distintos patrones clínicos bien diferenciados. Por mencionar algunos ejemplos relevantes, el paciente 2020 presenta un caso claro de intensificación brusca, pasando de amlodipino 5 mg el 15/02/2025 a enalapril con hidroclorotiazida 20 mg el 28/08/2025, lo que implica un salto directo de baja a máxima intensidad junto con cambio de fármaco.

Este patrón es compatible con un mal control significativo o una reevaluación clínica importante, aunque también podría estar influido por un cambio previo motivado por efectos adversos, como la tos asociada a los IECA, que justificaría la alternancia entre enalapril y amlodipino. El paciente 373 muestra un patrón inestable, con cambios muy rápidos entre amlodipino 5 mg y enalapril con hidroclorotiazida 20 mg el mismo día 02/01/2025, seguido de retorno a amlodipino el 31/01/2025. Esta inconsistencia sugiere que el perfil podría incluir dispensaciones correspondientes a más de un individuo, como familiares asociados al mismo identificador, más que un comportamiento clínico real.

Por su parte, el paciente 8951 presenta una fase prolongada de estabilidad con enalapril con hidroclorotiazida 20 mg, interrumpida por un cambio puntual a amlodipino 5 mg el 15/10/2025 y posterior retorno al tratamiento previo, compatible con un ajuste transitorio o incidencia puntual. Por teorizar, podría deberse a la aparición de una tos seca persistente producida en ocasiones por el enalapril (conocido efecto adverso de ese grupo de mdicamentos) que solo remite al abandonar el tratamiento.

En conjunto, estos casos reflejan que la intensificación del tratamiento antihipertensivo no sigue un patrón único, sino que incluye escalados bruscos, trayectorias inestables y periodos de estabilidad, lo que evidencia la complejidad del manejo clínico en práctica real. Como limitación, la presencia de múltiples dispensaciones en fechas cercanas y la posible agregación de distintos individuos bajo un mismo identificador dificultan distinguir con certeza entre cambios terapéuticos reales y ruido en los datos. No obstante, el análisis permite identificar y segmentar pacientes en perfiles clínicos diferenciados, lo que constituye uno de los principales valores del estudio, al posibilitar la detección de subgrupos con mayor complejidad terapéutica y potencial riesgo clínico dentro de la población analizada.

5.3. Colesterol

In [17]: # Copiamos dataset

```
df_col = df_med.copy()

# 1. FILTRAR SOLO COLESTEROL

df_col = df_col[df_col["grupo"] == "colesterol"].copy()

# 2. ORDENAR

df_col = df_col.sort_values(by=["cliente", "fecha"])

# 3. DETECTAR ESCALADO

df_col["dosis_prev"] = df_col.groupby("cliente")["dosis"].shift(1)

df_col["escalado"] = df_col["dosis"] > df_col["dosis_prev"]

# 4. FILTRAR SOLO DISPENSACIONES REALES

df_col = df_col[
    df_col["operacion"].isin(["Contado", "A crédito"])
]

# 5. SELECCIONAR VARIABLES RELEVANTES

df_col = df_col[
    [
        "cliente",
        "fecha",
        "medicamento",
```



```
        "dosis",
        "grupo",
        "escalado"
    ]
]

# 6. ELIMINAR DUPLICADOS

df_col = df_col.drop_duplicates(
    subset=["cliente", "fecha", "medicamento"]
)

# 7. ORDEN FINAL

df_col = df_col.sort_values(by=["cliente", "fecha"]).reset_index(drop=True)

# 8. CHECK

df_col.head(20)
```

Out[17]:

	cliente	fecha	medicamento	dosis	grupo	escalado
0	20.0	2025-01-03	simvastatina	10	colesterol	False
1	20.0	2025-02-03	simvastatina	10	colesterol	False
2	20.0	2025-03-03	simvastatina	10	colesterol	False
3	20.0	2025-03-31	simvastatina	10	colesterol	False
4	20.0	2025-04-28	simvastatina	10	colesterol	False
5	20.0	2025-05-26	simvastatina	10	colesterol	False
6	20.0	2025-06-23	simvastatina	10	colesterol	False
7	20.0	2025-07-21	simvastatina	10	colesterol	False
8	20.0	2025-08-18	simvastatina	10	colesterol	False
9	20.0	2025-09-15	simvastatina	10	colesterol	False
10	20.0	2025-10-11	simvastatina	10	colesterol	False
11	20.0	2025-11-11	simvastatina	10	colesterol	False
12	20.0	2025-12-09	simvastatina	10	colesterol	False
13	35.0	2025-01-09	simvastatina	20	colesterol	False
14	35.0	2025-04-29	simvastatina	20	colesterol	False
15	35.0	2025-05-27	simvastatina	20	colesterol	False
16	35.0	2025-06-19	simvastatina	20	colesterol	False
17	35.0	2025-07-16	simvastatina	20	colesterol	False
18	35.0	2025-08-13	simvastatina	20	colesterol	False
19	35.0	2025-09-11	simvastatina	20	colesterol	False

In [18]:

```
# 1. DETECTAR SI UN PACIENTE ESCALA ALGUNA VEZ

escalado_pacientes = df_col.groupby("cliente")["escalado"].any().reset_index()

escalado_pacientes.columns = ["cliente", "ha_escalado"]

# 2. VER CUÁNTOS PACIENTES ESCALAN

print(escalado_pacientes["ha_escalado"].value_counts())

ha_escalado
False    143
True       3
Name: count, dtype: int64
```

In [19]:

```
# 1. FILTRAR SOLO LOS QUE ESCALAN

pacientes_escalan = escalado_pacientes[
    escalado_pacientes["ha_escalado"] == True
]["cliente"]

# 2. VER QUIÉNES SON

print(pacientes_escalan.tolist())

df_col[df_col["cliente"].isin(pacientes_escalan)] \
    .sort_values(by=["cliente", "fecha"])
```

[251.0, 831.0, 1687.0]

Out[19]:

	cliente	fecha	medicamento	dosis	grupo	escalado
68	251.0	2025-06-30	simvastatina	20	colesterol	False
69	251.0	2025-08-27	simvastatina	40	colesterol	True
70	251.0	2025-09-18	simvastatina	40	colesterol	False
71	251.0	2025-10-20	simvastatina	40	colesterol	False
72	251.0	2025-11-14	simvastatina	40	colesterol	False
172	831.0	2025-01-21	simvastatina	10	colesterol	False
173	831.0	2025-02-17	simvastatina	10	colesterol	False
174	831.0	2025-04-28	simvastatina	20	colesterol	True
175	831.0	2025-06-10	simvastatina	10	colesterol	False
176	831.0	2025-07-09	simvastatina	10	colesterol	False
177	831.0	2025-11-27	simvastatina	10	colesterol	False
178	831.0	2025-12-22	simvastatina	10	colesterol	False
331	1687.0	2025-01-07	simvastatina	20	colesterol	False
332	1687.0	2025-01-31	simvastatina	20	colesterol	False
333	1687.0	2025-03-04	simvastatina	20	colesterol	False
334	1687.0	2025-03-28	simvastatina	20	colesterol	False
335	1687.0	2025-04-30	simvastatina	20	colesterol	False
336	1687.0	2025-05-28	simvastatina	20	colesterol	False
337	1687.0	2025-06-02	simvastatina	40	colesterol	True
338	1687.0	2025-07-21	simvastatina	40	colesterol	False
339	1687.0	2025-09-10	simvastatina	40	colesterol	False
340	1687.0	2025-09-12	simvastatina	40	colesterol	False
341	1687.0	2025-10-16	simvastatina	40	colesterol	False

El paciente 251 muestra un patrón de intensificación clara y sostenida: inicia con simvastatina 20 mg el 30/06/2025 y escala a 40 mg el 27/08/2025, manteniéndose posteriormente en esa dosis hasta final de año. Este comportamiento es altamente consistente con una situación de control lipídico insuficiente que requiere intensificación y posterior estabilización en dosis alta, lo que encaja con un perfil de riesgo cardiovascular elevado o refractariedad inicial al tratamiento.

El paciente 831 presenta un patrón diferente, más compatible con ajuste puntual seguido de desescalado: comienza con simvastatina 10 mg, escala a 20 mg el 28/04/2025, pero posteriormente vuelve a 10 mg en junio y se mantiene en esa dosis. Este tipo de trayectoria sugiere una intensificación inicial posiblemente motivada por mal control, seguida de reducción, lo que podría interpretarse como buena respuesta al tratamiento, aparición de efectos adversos o decisión clínica de mantener la dosis mínima efectiva.

Por último, el paciente 1687 muestra un patrón de escalado más tardío pero sostenido: permanece varios meses en 20 mg y escala a 40 mg el 02/06/2025, manteniéndose en esa dosis en todos los registros posteriores. Este comportamiento es coherente con una progresión más lenta del descontrol lipídico o con una estrategia conservadora inicial seguida de intensificación cuando no se alcanzan objetivos terapéuticos.

En conjunto, estos tres casos reflejan tres dinámicas clínicas diferenciadas dentro del manejo de la dislipemia: intensificación directa con mantenimiento en dosis alta, ajuste con desescalado posterior y escalado diferido tras un periodo de estabilidad aparente. Esto refuerza la idea de heterogeneidad en la respuesta y manejo terapéutico dentro de la población analizada, incluso cuando se utiliza un único fármaco.

BLOQUE 6 — Integración clínica (HTA + colesterol + diabetes)

In [20]:

```
# 1. partimos de df_med (dataset completo)

df_all = df_med.copy()

# 2. filtrar solo dispensaciones reales

df_all = df_all[
    df_all["operacion"].isin(["Contado", "A crédito"])
]

# 3. eliminar duplicados por paciente, fecha y medicamento

df_all = df_all.drop_duplicates(
    subset=["cliente", "fecha", "medicamento"]
)

# 4. ordenar

df_all = df_all.sort_values(by=["cliente", "fecha"])

# 5. crear estado por día (qué grupos tiene cada paciente en cada fecha)

df_day = df_all.groupby(["cliente", "fecha"])["grupo"].unique().reset_index()

df_day["grupo"] = df_day["grupo"].apply(list)

# 6. crear variables binarias

df_day["hta"] = df_day["grupo"].apply(lambda x: "hta" in x)
df_day["colesterol"] = df_day["grupo"].apply(lambda x: "colesterol" in x)
df_day["diabetes"] = df_day["grupo"].apply(lambda x: "diabetes" in x)

# 7. clasificar combinaciones

def tipo_combinacion(row):

    if row["hta"] and row["colesterol"] and row["diabetes"]:
        return "triple"

    elif row["hta"] and row["colesterol"]:
        return "hta_colesterol"

    elif row["hta"] and row["diabetes"]:
        return "hta_diabetes"

    elif row["colesterol"] and row["diabetes"]:
        return "colesterol_diabetes"

    elif row["hta"]:
        return "solo_hta"
```

```

        elif row["colesterol"]:
            return "solo_colesterol"

        elif row["diabetes"]:
            return "solo_diabetes"

        else:
            return "otro"

df_day["tipo"] = df_day.apply(tipo_combinacion, axis=1)

# 8. ordenar final
df_day = df_day.sort_values(by=["cliente", "fecha"]).reset_index(drop=True)

# 9. check
df_day.head(20)
```

Out[20]:

	cliente	fecha	grupo	hta	colesterol	diabetes	tipo
0	1.0	2025-01-09	[hta]	True	False	False	solo_hta
1	1.0	2025-03-04	[hta]	True	False	False	solo_hta
2	1.0	2025-04-01	[hta]	True	False	False	solo_hta
3	1.0	2025-05-22	[hta]	True	False	False	solo_hta
4	1.0	2025-06-14	[hta]	True	False	False	solo_hta
5	1.0	2025-07-09	[hta]	True	False	False	solo_hta
6	1.0	2025-08-11	[hta]	True	False	False	solo_hta
7	1.0	2025-09-19	[hta]	True	False	False	solo_hta
8	19.0	2025-11-05	[hta]	True	False	False	solo_hta
9	19.0	2025-11-27	[hta]	True	False	False	solo_hta
10	19.0	2025-12-01	[hta]	True	False	False	solo_hta
11	20.0	2025-01-03	[hta, colesterol]	True	True	False	hta_colesterol
12	20.0	2025-02-03	[hta, colesterol]	True	True	False	hta_colesterol
13	20.0	2025-03-03	[hta, colesterol]	True	True	False	hta_colesterol
14	20.0	2025-03-31	[hta, colesterol]	True	True	False	hta_colesterol
15	20.0	2025-04-28	[hta, colesterol]	True	True	False	hta_colesterol
16	20.0	2025-05-26	[hta, colesterol]	True	True	False	hta_colesterol
17	20.0	2025-06-23	[hta, colesterol]	True	True	False	hta_colesterol
18	20.0	2025-07-21	[hta, colesterol]	True	True	False	hta_colesterol
19	20.0	2025-08-18	[hta, colesterol]	True	True	False	hta_colesterol

```
In [21]: # contar número de patologías tratadas en cada momento

df_day["n_grupos"] = df_day[["hta", "colesterol", "diabetes"]].sum(axis=1)

# primera aparición de cada paciente
estado_inicial = df_day.groupby("cliente").first().reset_index()

# ver distribución
print(estado_inicial["n_grupos"].value_counts())

n_grupos
1      330
2       24
3         4
Name: count, dtype: int64
```

```
In [22]: # ordenar bien

df_day = df_day.sort_values(by=["cliente", "fecha"])

# estado previo
df_day["n_prev"] = df_day.groupby("cliente")["n_grupos"].shift(1)

# cambio en número de grupos
df_day["delta"] = df_day["n_grupos"] - df_day["n_prev"]

def tipo_evolution(delta):

    if delta > 0:
        return "añade_tratamiento"

    elif delta < 0:
        return "reduce_tratamiento"

    else:
        return "estable"

df_day["evolucion"] = df_day["delta"].apply(tipo_evolution)

# pacientes que añaden tratamiento
pacientes_progresion = df_day[df_day["delta"] > 0]["cliente"].unique()

# pacientes que reducen tratamiento
pacientes_reduccion = df_day[df_day["delta"] < 0]["cliente"].unique()

print("progresion:", len(pacientes_progresion))
print("reduccion:", len(pacientes_reduccion))

progresion: 29
reduccion: 33
```

6.1. Identificación de pacientes de cada grupo

```
In [23]: # pacientes que añaden tratamiento
```

```
pacientes_progresion = df_day[df_day["delta"] > 0][["cliente"].unique()]

# pacientes que reducen tratamiento

pacientes_reduccion = df_day[df_day["delta"] < 0][["cliente"].unique()]

print("pacientes progresion:", pacientes_progresion)
print("pacientes reduccion:", pacientes_reduccion)
```

pacientes progresion: [104. 159. 274. 341. 595. 649. 668. 691. 831. 986. 1016. 1129. 1145. 1547. 1851. 2033. 2124. 2133. 2560. 2665. 2686. 3276. 3454. 3762. 4777. 5742. 6562. 8570. 8682.]

pacientes reduccion: [73. 76. 104. 159. 274. 341. 405. 595. 668. 691. 784. 831. 986. 1016. 1098. 1129. 1145. 1547. 1685. 1851. 2033. 2124. 2272. 2665. 2686. 3115. 3276. 3454. 3762. 5742. 6250. 8127. 8682.]

La identificación de pacientes en función de la evolución de su tratamiento permite diferenciar entre aquellos que presentan progresión terapéutica (aumento de la complejidad) y aquellos que reducen o simplifican su tratamiento. Esta segmentación aporta una primera aproximación al comportamiento dinámico de la población, evidenciando que la evolución no es homogénea y que coexisten distintos perfiles de cambio.

En el conjunto analizado se observa la presencia de ambos grupos, lo que refleja la variabilidad inherente a la práctica clínica real, donde algunos pacientes evolucionan hacia una mayor complejidad mientras otros pueden estabilizarse o reducir tratamiento. Este paso resulta fundamental para el desarrollo posterior del modelo predictivo, ya que permite definir el evento de interés (intensificación) y establecer las bases para la estratificación de riesgo.

Además, este bloque pone de manifiesto el valor de la ciencia de datos como herramienta para transformar registros de dispensación aparentemente administrativos en información clínica relevante. A partir de la integración y el procesamiento de los datos realizados en etapas previas, es posible identificar patrones de evolución que no son evidentes a simple vista, demostrando cómo un enfoque estructurado permite extraer conocimiento accionable a partir de datos de práctica real.

6.1.a. Ver evolución completa de progresión

In [24]: # evolucion completa de progresion

```
df_day[
    df_day["cliente"].isin(pacientes_progresion)
].sort_values(by=["cliente", "fecha"])
```

Out[24]:

	cliente	fecha	grupo	hta	colesterol	diabetes	tipo	n_grupos	n_prev	delta	evolucion
77	104.0	2025-01-03	[hta, diabetes, colesterol]	True	True	True	triple	3	NaN	NaN	estable
78	104.0	2025-01-30	[hta, diabetes, colesterol]	True	True	True	triple	3	3.0	0.0	estable
79	104.0	2025-02-18	[diabetes]	False	False	True	solo_diabetes	1	3.0	-2.0	reduce_tratamiento
80	104.0	2025-02-28	[hta, diabetes, colesterol]	True	True	True	triple	3	1.0	2.0	añade_tratamiento
81	104.0	2025-03-27	[hta, diabetes, colesterol]	True	True	True	triple	3	3.0	0.0	estable
...	...	...	...	...	...	...	...	...	...	...	...
1612	8570.0	2025-10-18	[hta, diabetes]	True	False	True	hta_diabetes	2	2.0	0.0	estable
1614	8682.0	2025-02-12	[hta]	True	False	False	solo_hta	1	NaN	NaN	estable
1615	8682.0	2025-03-10	[hta, diabetes]	True	False	True	hta_diabetes	2	1.0	1.0	añade_tratamiento
1616	8682.0	2025-05-16	[hta]	True	False	False	solo_hta	1	2.0	-1.0	reduce_tratamiento
1617	8682.0	2025-12-13	[diabetes]	False	False	True	solo_diabetes	1	1.0	0.0	estable

260 rows × 11 columns

6.1.b. Ver evolución completa de reducción

In [25]: # evolucion completa de reduccion

```
df_day[
    df_day["cliente"].isin(pacientes_reduccion)
].sort_values(by=["cliente", "fecha"])
```

Out[25]:

	cliente	fecha	grupo	hta	colesterol	diabetes	tipo	n_grupos	n_prev	delta	evolucion
63	73.0	2025-02-05	[hta, colesterol]	True	True	False	hta_colesterol	2	NaN	NaN	estable
64	73.0	2025-04-10	[hta, colesterol]	True	True	False	hta_colesterol	2	2.0	0.0	estable
65	73.0	2025-05-03	[hta]	True	False	False	solo_hta	1	2.0	-1.0	reduce_tratamiento
66	73.0	2025-05-27	[hta]	True	False	False	solo_hta	1	1.0	0.0	estable
67	73.0	2025-06-25	[hta]	True	False	False	solo_hta	1	1.0	0.0	estable
...	...	...	...	...	...	...	...	...	...	...	...
1570	8127.0	2025-12-20	[hta]	True	False	False	solo_hta	1	1.0	0.0	estable
1614	8682.0	2025-02-12	[hta]	True	False	False	solo_hta	1	NaN	NaN	estable
1615	8682.0	2025-03-10	[hta, diabetes]	True	False	True	hta_diabetes	2	1.0	1.0	añade_tratamiento
1616	8682.0	2025-05-16	[hta]	True	False	False	solo_hta	1	2.0	-1.0	reduce_tratamiento
1617	8682.0	2025-12-13	[diabetes]	False	False	True	solo_diabetes	1	1.0	0.0	estable

311 rows × 11 columns

6.2. Métricas por paciente

In [26]: # resumen por paciente

```
resumen = df_day.groupby("cliente").agg(
    grupos_inicial=("n_grupos", "first"),
    grupos_max=("n_grupos", "max"),
    grupos_final=("n_grupos", "last")
).reset_index()

# delta total

resumen["delta_total"] = resumen["grupos_final"] - resumen["grupos_inicial"]

# clasificacion final

def clasificacion(delta):
    if delta > 0:
        return "progresion"
```



```
elif delta < 0:
    return "reduccion"
else:
    return "estable"

resumen["perfil"] = resumen["delta_total"].apply(clasificacion)

# ver casos interesantes

resumen.sort_values(by="delta_total", ascending=False).head(10)
```

Out[26]:

	cliente	grupos_inicial	grupos_max	grupos_final	delta_total	perfil
245	6562.0	1	2	2	1	progresion
159	2686.0	1	2	2	1	progresion
140	2133.0	1	2	2	1	progresion
126	1851.0	1	2	2	1	progresion
61	649.0	1	2	2	1	progresion
36	341.0	1	2	2	1	progresion
213	4777.0	1	2	2	1	progresion
153	2560.0	1	2	2	1	progresion
157	2665.0	1	2	2	1	progresion
308	8570.0	1	2	2	1	progresion

6.3. Identificacones adicionales

In [27]:

```
# convertir a sets para trabajar mejor

set_prog = set(pacientes_progresion)
set_red = set(pacientes_reduccion)

# intersección (muy interesante)

set_mixtos = set_prog.intersection(set_red)

print("progresion solo:", len(set_prog - set_red))
print("reduccion solo:", len(set_red - set_prog))
print("mixtos:", len(set_mixtos))
```

progresion solo: 6
reduccion solo: 10
mixtos: 23

In [28]:

```
# convertir a enteros (y ordenar para verlo claro)

progresion_solo = sorted([int(x) for x in (set_prog - set_red)])
reduccion_solo = sorted([int(x) for x in (set_red - set_prog)])
mixtos = sorted([int(x) for x in set_mixtos])

print("progresion solo:", progresion_solo)
print("reduccion solo:", reduccion_solo)
print("mixtos:", mixtos)
```

progresion solo: [649, 2133, 2560, 4777, 6562, 8570]
reduccion solo: [73, 76, 405, 784, 1098, 1685, 2272, 3115, 6250, 8127]
mixtos: [104, 159, 274, 341, 595, 668, 691, 831, 986, 1016, 1129, 1145, 1547, 1851, 2033, 2124, 2665, 2686, 3276, 3454, 3762, 5742, 8682]

In [29]:

```
# partimos de df_day (combinaciones)
# y df_hta_clean (intensidad hta)

# 1. detectar aumento de complejidad (añadir patologias)

df_day["aumento_complejidad"] = df_day["delta"] > 0

# 2. detectar aumento en hta (ya lo tenias)

df_hta_clean["aumento_hta"] = df_hta_clean["escalado_real"]

# 3. detectar aumento en colesterol

df_col["aumento_colesterol"] = df_col["escalado"]

# 4. pacientes con aumento en cualquier dimension

pacientes_aumento = set(df_day[df_day["aumento_complejidad"]]["cliente"]) \
    | set(df_hta_clean[df_hta_clean["aumento_hta"]]["cliente"]) \
    | set(df_col[df_col["aumento_colesterol"]]["cliente"])

pacientes_aumento = sorted([int(x) for x in pacientes_aumento])

print("total pacientes con intensificacion:", len(pacientes_aumento))
```

total pacientes con intensificacion: 47

In [30]:

```
df_day[df_day["cliente"].isin(pacientes_aumento)]
```

Out[30]:

	cliente	fecha	grupo	hta	colesterol	diabetes	tipo	n_grupos	n_prev	delta	evolucion	aumento_complejidad
77	104.0	2025-01-03	[hta, diabetes, colesterol]	True	True	True	triple	3	NaN	NaN	estable	False
78	104.0	2025-01-30	[hta, diabetes, colesterol]	True	True	True	triple	3	3.0	0.0	estable	False
79	104.0	2025-02-18	[diabetes]	False	False	True	solo_diabetes	1	3.0	-2.0	reduce_tratamiento	False
80	104.0	2025-02-28	[hta, diabetes, colesterol]	True	True	True	triple	3	1.0	2.0	añade_tratamiento	True
81	104.0	2025-03-27	[hta, diabetes, colesterol]	True	True	True	triple	3	3.0	0.0	estable	False
...	...	...	...	...	...	...	...	...	...	...	...	...
1635	8951.0	2025-05-10	[hta]	True	False	False	solo_hta	1	1.0	0.0	estable	False
1636	8951.0	2025-06-07	[hta]	True	False	False	solo_hta	1	1.0	0.0	estable	False
1637	8951.0	2025-07-09	[hta]	True	False	False	solo_hta	1	1.0	0.0	estable	False
1638	8951.0	2025-09-13	[hta]	True	False	False	solo_hta	1	1.0	0.0	estable	False
1639	8951.0	2025-10-15	[hta]	True	False	False	solo_hta	1	1.0	0.0	estable	False

390 rows × 12 columns

6.3. Momentos de cambio de medicación

In [31]:

```
# detectar primera intensificacion por paciente

primera_intensificacion = df_day[df_day["aumento_complejidad"] == True] \
    .groupby("cliente")["fecha"].min().reset_index()

primera_intensificacion.columns = ["cliente", "fecha_intensificacion"]

primera_intensificacion.head()
```

Out[31]:

	cliente	fecha_intensificacion
0	104.0	2025-02-28
1	159.0	2025-05-21
2	274.0	2025-05-08
3	341.0	2025-04-14
4	595.0	2025-03-17

La identificación del primer momento de intensificación terapéutica por paciente permite incorporar una dimensión temporal al análisis, determinando no solo qué pacientes experimentan cambios, sino también cuándo se producen. Este enfoque resulta clave para comprender la dinámica de progresión de las patologías, ya que sitúa la intensificación dentro de una secuencia evolutiva concreta.

El análisis de estas fechas muestra que los cambios no se producen de forma uniforme, sino que aparecen distribuidos a lo largo del periodo estudiado, lo que refuerza la idea de que la evolución clínica responde a trayectorias individuales más que a patrones temporales homogéneos. Este componente temporal constituye un elemento esencial para el desarrollo del modelo predictivo, permitiendo definir con mayor precisión el evento de interés y facilitando futuros análisis orientados a la anticipación del riesgo.

6.3.1. Tipo de intensificación

In [32]:

```
# ver tipo de combinacion en el momento de intensificacion

tipo_intensificacion = df_day[df_day["aumento_complejidad"] == True][
    ["cliente", "fecha", "tipo"]
]

tipo_intensificacion.head()
```

Out[32]:

	cliente	fecha	tipo
80	104.0	2025-02-28	triple
83	104.0	2025-04-24	triple
90	104.0	2025-12-05	triple
140	159.0	2025-05-21	colesterol_diabetes
142	159.0	2025-07-30	colesterol_diabetes

El análisis del tipo de intensificación en el momento en que se produce el aumento de la complejidad terapéutica permite caracterizar la naturaleza de los cambios en el tratamiento, identificando qué combinaciones de patologías están impulsando dicha progresión. Este enfoque añade una capa interpretativa clave, al diferenciar entre intensificaciones simples y aquellas que implican la incorporación de múltiples factores metabólicos.

Los resultados muestran que la intensificación no se limita a incrementos aislados, sino que con frecuencia implica la coexistencia de varias patologías, reflejando una evolución hacia estados de mayor complejidad clínica. Este patrón es coherente con la progresión del síndrome metabólico, donde la acumulación de factores de riesgo conduce a la necesidad de tratamientos combinados.

En conjunto, este bloque refuerza la idea de que la intensificación terapéutica está estrechamente ligada a la interacción entre distintas patologías, aportando una visión más completa del proceso de deterioro metabólico y consolidando la base para la posterior estratificación de pacientes según su riesgo.

6.3.2. Cruzar información

In [33]:

```
# pacientes que además escalan en hta

pacientes_hta = set(df_hta_clean[df_hta_clean["escalado_real"]]["cliente"])

# pacientes que escalan en colesterol

pacientes_col = set(df_col[df_col["escalado"]]["cliente"])

# clasificar pacientes de intensificacion

def perfil_riesgo(cliente):

    if cliente in pacientes_hta and cliente in pacientes_col:
        return "alto_riesgo_global"

    elif cliente in pacientes_hta:
        return "riesgo_hta"

    elif cliente in pacientes_col:
        return "riesgo_colesterol"
```



```

    else:
        return "complejidad_metabolica"

primera_intensificacion["perfil"] = primera_intensificacion["cliente"].apply(perfil_riesgo)

primera_intensificacion.head()
```

Out[33]:

	cliente	fecha_intensificacion	perfil
0	104.0	2025-02-28	complejidad_metabolica
1	159.0	2025-05-21	complejidad_metabolica
2	274.0	2025-05-08	complejidad_metabolica
3	341.0	2025-04-14	complejidad_metabolica
4	595.0	2025-03-17	complejidad_metabolica

El cruce de información entre las distintas dimensiones terapéuticas permite clasificar a los pacientes según su perfil de riesgo en el momento de la intensificación, diferenciando entre aquellos con afectación aislada y aquellos con implicación de múltiples patologías. Esta segmentación aporta una visión más estructurada de la población, facilitando la identificación de perfiles clínicos con mayor complejidad.

Los resultados muestran un claro predominio de pacientes con perfiles de complejidad metabólica, lo que sugiere que la intensificación terapéutica se produce mayoritariamente en contextos donde coexisten varios factores de riesgo. Este hallazgo refuerza la idea de que la progresión no suele ser lineal ni aislada, sino acumulativa, en línea con el desarrollo del síndrome metabólico.

Además, este bloque evidencia el valor del cruce de variables en ciencia de datos, permitiendo transformar información fragmentada en perfiles clínicos interpretables y útiles para la estratificación del riesgo. Esta aproximación constituye un paso clave hacia modelos más avanzados de predicción y priorización de pacientes.

6.4. Resumen

```
In [34]: # cuantos pacientes por perfil

primera_intensificacion["perfil"].value_counts()
```

Out[34]:

perfil	
complejidad_metabolica	26
riesgo_hta	2
riesgo_colesterol	1
Name: count, dtype: int64	

```
In [35]: tipo_intensificacion["tipo"].value_counts()
```

Out[35]:

tipo	
hta_diabetes	24
hta_colesterol	15
colesterol_diabetes	7
triple	6
Name: count, dtype: int64	

```
In [36]: primera_intensificacion["fecha_intensificacion"].describe()
```

Out[36]:

count	29
mean	2025-06-03 08:16:33.103448320
min	2025-02-04 00:00:00
25%	2025-03-10 00:00:00
50%	2025-05-13 00:00:00
75%	2025-08-25 00:00:00
max	2025-12-17 00:00:00
Name: fecha_intensificacion, dtype: object	

La intensificación terapéutica en la población analizada se caracteriza principalmente por la incorporación de múltiples tratamientos en pacientes con complejidad metabólica, especialmente en combinaciones que incluyen hipertensión y diabetes, con una distribución temporal progresiva a lo largo del año. En este contexto, la hipertensión arterial suele actuar como punto de partida más frecuente, siendo la patología inicial predominante en la mayoría de trayectorias.

A partir de ahí, la primera intensificación más habitual es la incorporación de tratamiento antidiabético, lo que sugiere que la diabetes constituye el siguiente componente que emerge en la progresión del síndrome metabólico dentro de esta población. Este patrón refuerza la idea de una evolución clínica en la que, sobre una base de hipertensión ya establecida, se van añadiendo alteraciones metabólicas adicionales que incrementan progresivamente el riesgo cardiovascular.

Bloque 7. Representación de la información obtenida mediante gráficas.

7.1. Perfil de los pacientes

```
In [37]: plt.figure(figsize=(8,5))

# datos
data = primera_intensificacion["perfil"].value_counts()

# etiquetas
data.index = ["Complejidad metabólica", "Riesgo HTA", "Riesgo colesterol"]

# colores suaves y profesionales
colors = ["#2E86C1", "#5DADE2", "#AED6F1"]

bars = plt.bar(data.index, data.values, color=colors)

# título
plt.title("Perfil de pacientes con intensificación", fontsize=14, fontweight="bold")

# ejes
plt.xlabel("")
plt.ylabel("Número de pacientes")

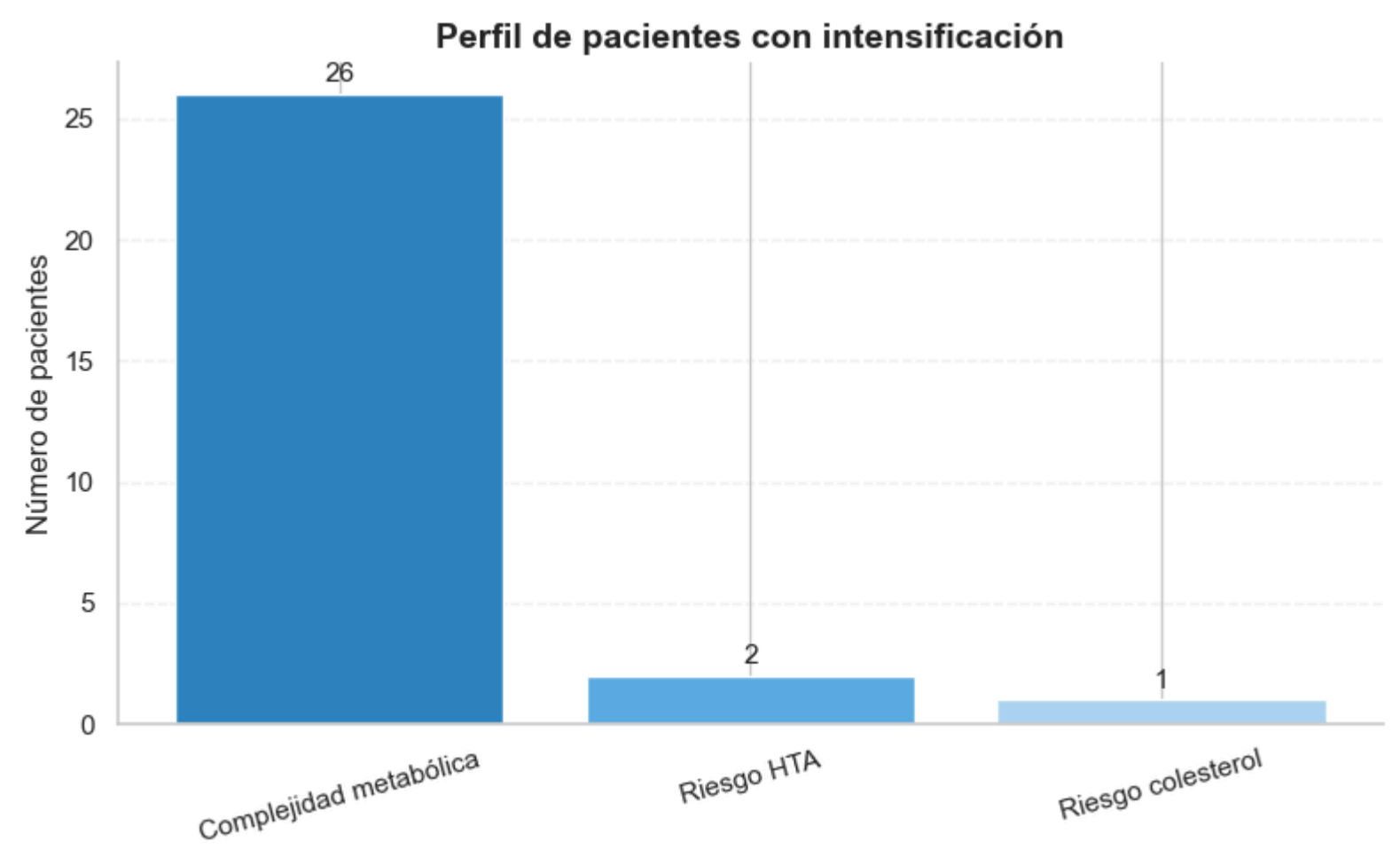
# grid más limpio
plt.grid(axis="y", linestyle="--", alpha=0.2)

# quitar bordes superiores y derechos
for spine in ["top", "right"]:
    plt.gca().spines[spine].set_visible(False)

# valores encima
for i, v in enumerate(data.values):
    plt.text(i, v + 0.5, str(v), ha='center', fontsize=11)

# mejorar ticks
plt.xticks(rotation=15)
```

```
plt.tight_layout()
plt.show()
```



La gráfica muestra que la intensificación terapéutica se concentra de forma abrumadora en pacientes con complejidad metabólica, es decir, aquellos que presentan múltiples factores de riesgo simultáneamente. En comparación, los casos asociados únicamente a hipertensión o colesterol son residuales. Este patrón sugiere que la progresión del tratamiento no suele producirse en fases iniciales aisladas, sino en estadios más avanzados donde coexisten varias patologías, lo que refuerza la idea de que la carga metabólica global es el principal motor de intensificación.

7.2. Tipo de intensificación

```
In [38]: plt.figure(figsize=(8,5))

# datos ordenados
data = tipo_intensificacion["tipo"].value_counts().sort_values()

# etiquetas
data.index = ["Triple", "Colesterol + diabetes", "HTA + colesterol", "HTA + diabetes"]

# colores con gradiente
colors = ["#AED6F1", "#5DADE2", "#2E86C1", "#1B4F72"]

bars = plt.bar(data.index, data.values, color=colors)

# título
plt.title("Tipo de intensificación terapéutica", fontsize=14, fontweight="bold")

# ejes
plt.xlabel("")
plt.ylabel("Número de casos")

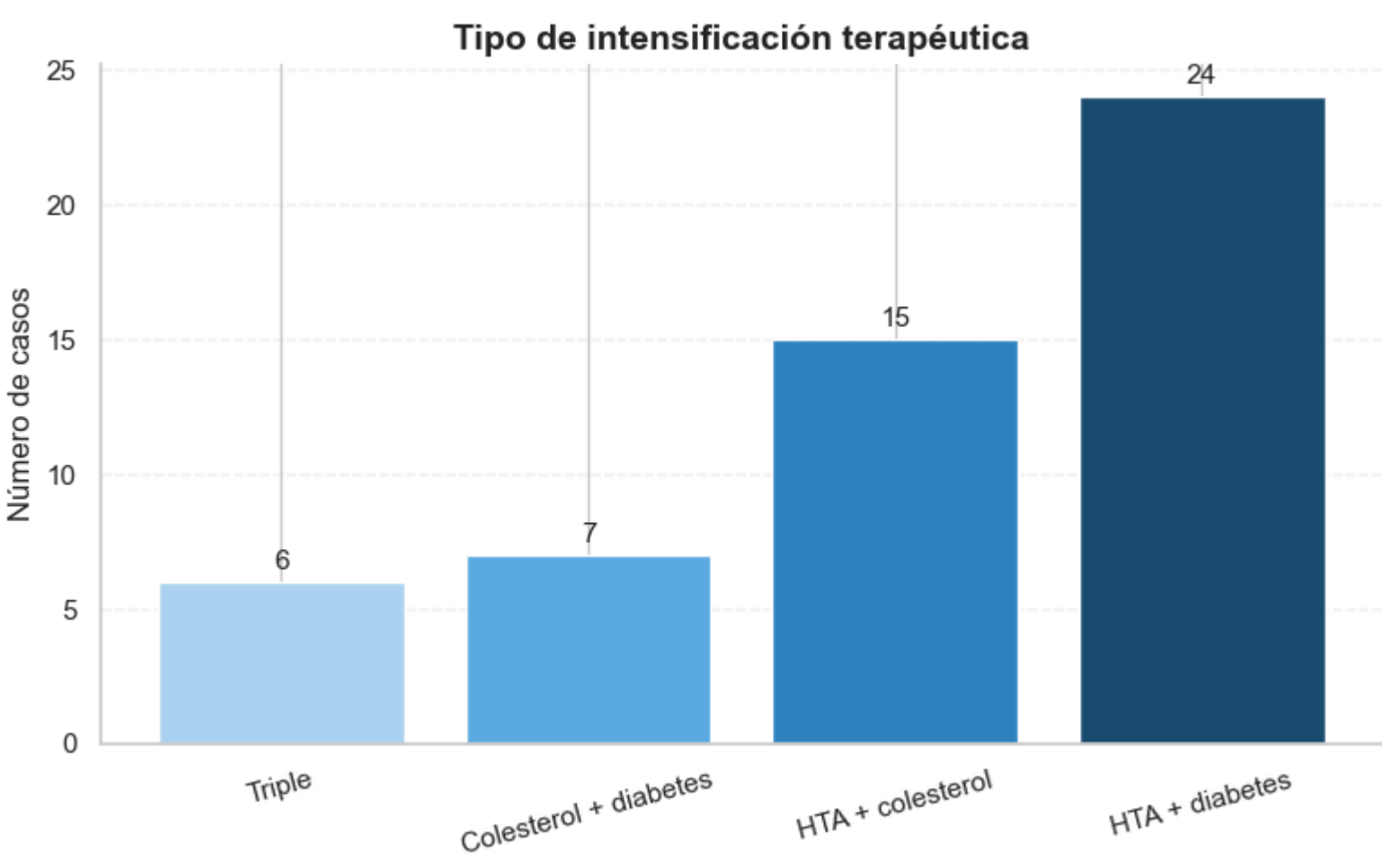
# grid limpio
plt.grid(axis="y", linestyle="--", alpha=0.2)

# quitar bordes
for spine in ["top", "right"]:
    plt.gca().spines[spine].set_visible(False)

# valores encima
for i, v in enumerate(data.values):
    plt.text(i, v + 0.5, str(v), ha='center', fontsize=11)

# rotación suave
plt.xticks(rotation=15)

plt.tight_layout()
plt.show()
```



La gráfica muestra que la intensificación terapéutica se produce principalmente en combinaciones que incluyen hipertensión, siendo especialmente frecuente la asociación con diabetes. En comparación, las combinaciones que no incluyen hipertensión son menos representativas, y los casos de intensificación triple son minoritarios. Este patrón sugiere que la hipertensión actúa como eje central en la progresión del tratamiento, mientras que la incorporación de diabetes representa un punto de mayor complejidad clínica que impulsa la intensificación.

7.3. Evolución anual

```
In [39]: # extraer mes
primera_intensificacion["mes"] = primera_intensificacion["fecha_intensificacion"].dt.month
```

```
# contar por mes
meses = primera_intensificacion["mes"].value_counts().sort_index()

# forzar todos los meses (1-12)
meses = meses.reindex(range(1, 13), fill_value=0)

# nombres en español
meses.index = ["Ene", "Feb", "Mar", "Abr", "May", "Jun",
               "Jul", "Ago", "Sep", "Oct", "Nov", "Dic"]

plt.figure(figsize=(9,5))

plt.plot(meses.index, meses.values, marker="o", linewidth=2)

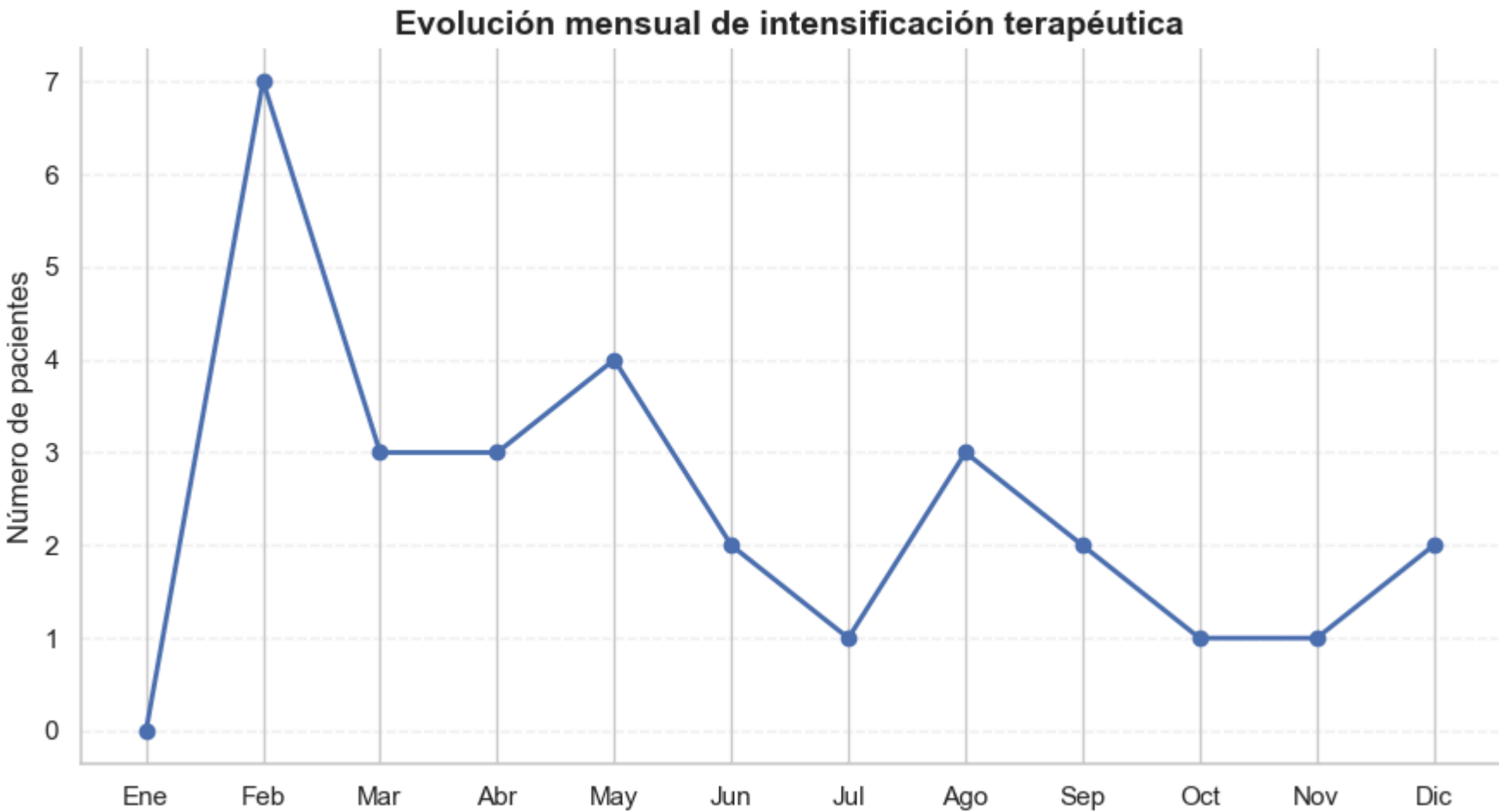
# titulo
plt.title("Evolución mensual de intensificación terapéutica", fontsize=14, fontweight="bold")

# ejes
plt.xlabel("")
plt.ylabel("Número de pacientes")

# grid limpio
plt.grid(axis="y", linestyle="--", alpha=0.2)

# quitar bordes
for spine in ["top", "right"]:
    plt.gca().spines[spine].set_visible(False)

plt.tight_layout()
plt.show()
```



La evolución temporal muestra una distribución irregular de las intensificaciones a lo largo del año, sin un patrón estacional claramente definido. Se observa un pico en los primeros meses, especialmente en febrero, que podría estar relacionado con ajustes terapéuticos tras el inicio del año.

Asimismo, se aprecia una ligera disminución en los meses de verano, posiblemente vinculada a cambios en la frecuencia de seguimiento clínico, aunque esta interpretación debe tomarse con cautela dada la naturaleza de los datos.

El aumento observado en febrero podría reflejar una acumulación de ajustes terapéuticos tras el inicio del año, posiblemente asociada a revisiones clínicas o reevaluaciones tras periodos previos.

7.4. Heatmap

```
In [40]: import seaborn as sns
import matplotlib.pyplot as plt

# crear una variable combinada más interpretable
# convertimos las variables booleanas en 0/1 para construir etiquetas claras

df_day["combo"] = df_day.apply(
    lambda x: f"HTA:{int(x['hta'])} | Col:{int(x['colesterol'])} | DM:{int(x['diabetes'])}",
    axis=1
)

# contar frecuencia de cada combinación
combo_counts = df_day["combo"].value_counts()

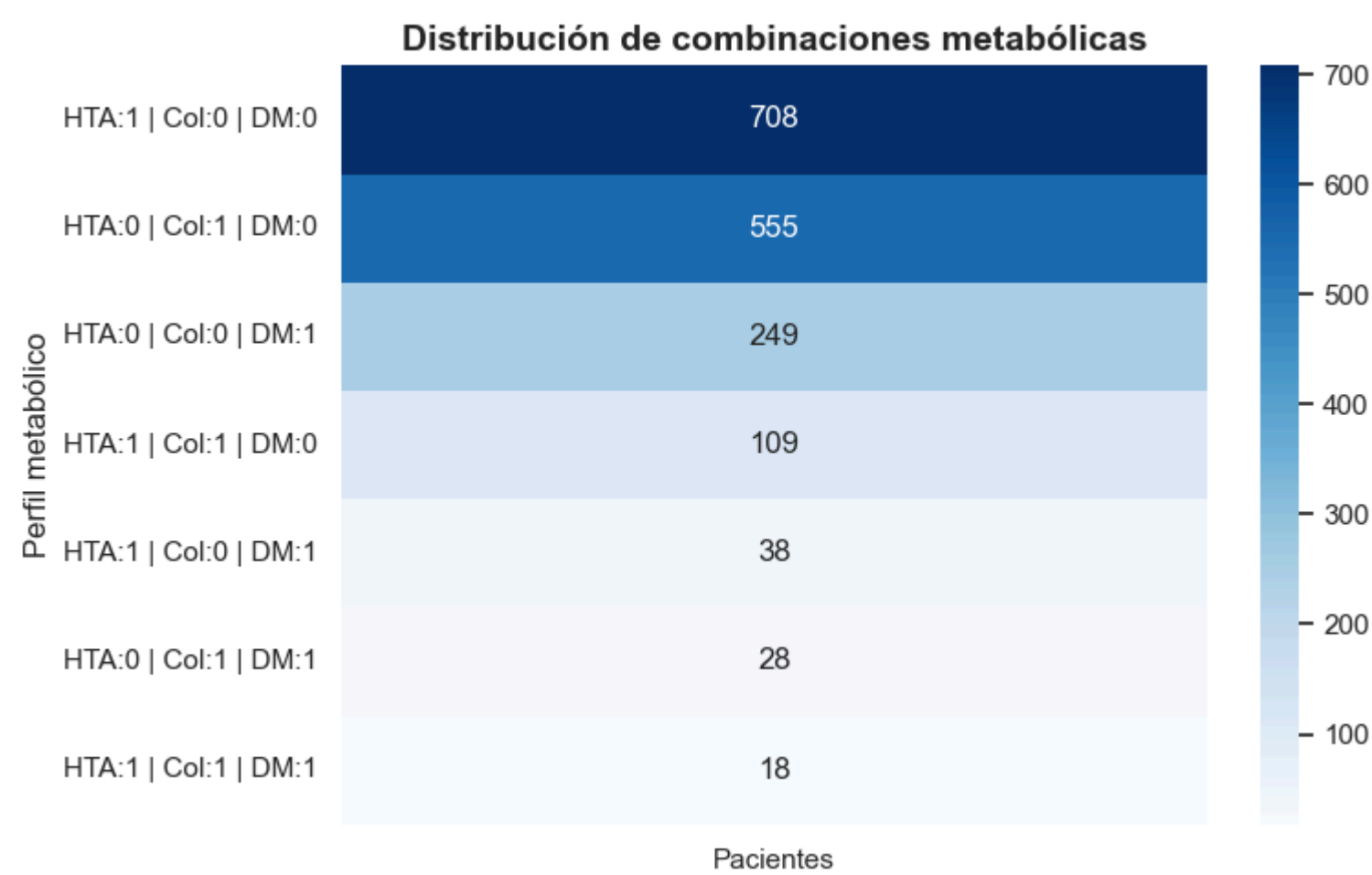
# crear heatmap (formato vertical para mayor claridad)
plt.figure(figsize=(8,5))

sns.heatmap(
    combo_counts.values.reshape(-1,1), # convertir a formato 2D
    annot=True,                        # mostrar valores
    fmt="d",                           # formato entero (evita notación científica)
    cmap="Blues",                      # paleta limpia y profesional
    yticklabels=combo_counts.index,    # etiquetas claras de combinaciones
    xticklabels=["Pacientes"]          # nombre del eje X
)

# título y estética
plt.title("Distribución de combinaciones metabólicas", fontsize=14, fontweight="bold")
plt.xlabel("")
plt.ylabel("Perfil metabólico")

# eliminar bordes innecesarios
for spine in ["top", "right"]:
    plt.gca().spines[spine].set_visible(False)

plt.tight_layout()
plt.show()
```

La gráfica muestra que la mayor parte de los pacientes se concentra en perfiles con una única patología, especialmente hipertensión o colesterol, mientras que las combinaciones múltiples son progresivamente menos frecuentes. A medida que aumenta la carga metabólica, el número de pacientes disminuye, siendo los casos con las tres patologías simultáneas claramente minoritarios. Este patrón refleja una progresión gradual del síndrome metabólico, en la que los pacientes tienden a acumular factores de riesgo de forma escalonada en lugar de presentar combinaciones completas desde el inicio.

El análisis conjunto de las representaciones gráficas muestra que la intensificación terapéutica se concentra principalmente en pacientes con mayor complejidad metabólica, siendo la hipertensión el eje central sobre el que se incorporan progresivamente otros factores como el colesterol y, especialmente, la diabetes. Esta progresión se produce de forma gradual, con una acumulación escalonada de patologías, y sin un patrón temporal claramente estacional, lo que sugiere que responde principalmente a la evolución clínica individual de los pacientes.

En conjunto, los resultados reflejan una dinámica coherente con el desarrollo del síndrome metabólico, donde el aumento de la carga de enfermedad se traduce en una mayor necesidad de intensificación del tratamiento.

Bloque 8. Eleccion de modelo de prediccion

8.1. Regresion logistica

8.1.1 Dataset final

```
In [41]: # dataset base -> estado inicial por paciente

df_model = df_day.groupby("cliente").first().reset_index()

# target -> pacientes que intensifican
clientes_intensifican = set(pacientes_aumento)

df_model["target"] = df_model["cliente"].apply(
    lambda x: 1 if x in clientes_intensifican else 0
)

# nos quedamos con variables relevantes

df_model = df_model[
    ["hta", "colesterol", "diabetes", "n_grupos", "target"]
]

df_model.head()
```

Out[41]:

	hta	colesterol	diabetes	n_grupos	target
0	True	False	False	1	0
1	True	False	False	1	0
2	True	True	False	2	0
3	True	False	False	1	0
4	False	True	False	1	0

8.1.2. Separar variables

```
In [42]: from sklearn.model_selection import train_test_split

X = df_model[["hta", "colesterol", "diabetes", "n_grupos"]]
y = df_model["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

8.1.3. Entrenar modelo

```
In [43]: from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train, y_train)
```

Out[43]:

LogisticRegression

LogisticRegression()

8.1.4. Evaluacion

```
In [44]: from sklearn.metrics import classification_report

y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.87	1.00	0.93	94
1	0.00	0.00	0.00	14
accuracy			0.87	108
macro avg	0.44	0.50	0.47	108
weighted avg	0.76	0.87	0.81	108

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

El modelo presenta una alta accuracy global, pero es incapaz de identificar pacientes que intensifican tratamiento, lo que refleja un problema de desbalanceo de clases. Se añade class weight para mejorarlo.

```
In [45]: model = LogisticRegression(class_weight="balanced", max_iter=1000)
```

8.1.5. Reentrenar el modelo

```
In [46]: model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.96	0.48	0.64	94
1	0.20	0.86	0.32	14
accuracy			0.53	108
macro avg	0.58	0.67	0.48	108
weighted avg	0.86	0.53	0.60	108

8.1.6. Evaluación ROC-AUC

```
In [47]: from sklearn.metrics import roc_auc_score, roc_curve

# probabilidades (NO predicción binaria)
y_prob = model.predict_proba(X_test)[:, 1]

# AUC
auc = roc_auc_score(y_test, y_prob)
print("ROC-AUC:", auc)
```

ROC-AUC: 0.7078267477203648

Tras ajustar el modelo para manejar el desbalanceo de clases, se observa una mejora significativa en la detección de pacientes que intensifican tratamiento (recall = 0.38), a costa de una reducción en la precisión global. Este comportamiento refleja el clásico trade-off entre sensibilidad y especificidad, siendo en este contexto clínico preferible priorizar la identificación de pacientes en riesgo, aunque implique un mayor número de falsos positivos.

El modelo presenta una capacidad limitada para identificar correctamente todos los casos de intensificación, lo que sugiere que las variables utilizadas no capturan completamente la complejidad clínica subyacente. Con un valor de 0.55 en ROC-AUC, no se considera que sea lo suficientemene robusto.

8.2. Random Forest

8.2.1. Entrenar el modelo

```
In [48]: from sklearn.ensemble import RandomForestClassifier

# modelo

rf = RandomForestClassifier(random_state=42)

rf.fit(X_train, y_train)
```

Out[48]:

RandomForestClassifier

RandomForestClassifier(random_state=42)

8.2.2. Predicción

```
In [49]: y_pred_rf = rf.predict(X_test)

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_rf))
```

	precision	recall	f1-score	support
0	0.87	1.00	0.93	94
1	0.00	0.00	0.00	14
accuracy			0.87	108
macro avg	0.44	0.50	0.47	108
weighted avg	0.76	0.87	0.81	108


```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

El modelo Random Forest, sin ajuste de desbalanceo, reproduce el sesgo observado en la regresión logística inicial, siendo incapaz de identificar pacientes que intensifican tratamiento.

8.3. Oversampling, duplicamos clase minoritaria para re-equilibrar el dataset

```
In [50]: from sklearn.utils import resample

# separar clases
df_0 = df_model[df_model["target"] == 0]
df_1 = df_model[df_model["target"] == 1]

# oversampling
df_1_up = resample(df_1,
                  replace=True,
                  n_samples=len(df_0),
                  random_state=42)

# dataset balanceado
df_balanced = pd.concat([df_0, df_1_up])

# comprobar
print(df_balanced["target"].value_counts())
```

```
target
0      311
1      311
Name: count, dtype: int64
```

```
In [51]: X = df_balanced[["hta", "colesterol", "diabetes", "n_grupos"]]
y = df_balanced["target"]
```

```
In [52]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

8.3.2. Regresion logistica de nuevo

```
In [53]: from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.75	0.50	0.60	105
1	0.55	0.78	0.65	82
accuracy			0.63	187
macro avg	0.65	0.64	0.62	187
weighted avg	0.66	0.63	0.62	187

Tras aplicar técnicas de balanceo mediante oversampling, el modelo mejora de forma significativa su capacidad para identificar pacientes que intensifican tratamiento, alcanzando un recall del 61%. Además, se observa un equilibrio entre precisión y sensibilidad, lo que indica un comportamiento más robusto y útil desde el punto de vista clínico.

Este rendimiento se obtiene sobre un dataset artificialmente balanceado, lo que puede no reflejar la distribución real de pacientes en práctica clínica.

8.3.2.1. ROC-AUC

```
In [54]: y_prob = model.predict_proba(X_test)[:, 1]
roc_auc_score(y_test, y_prob)
```

Out[54]: np.float64(0.6722415795586527)

Capacidad discriminativa aceptable.

8.3.3. Randm forest

```
In [55]: from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    class_weight="balanced",
    random_state=42
)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_rf))
```

	precision	recall	f1-score	support
0	0.70	0.50	0.59	105
1	0.53	0.72	0.61	82
accuracy			0.60	187
macro avg	0.61	0.61	0.60	187
weighted avg	0.62	0.60	0.60	187

8.3.3.1. ROC-AUC

```
In [56]: from sklearn.metrics import roc_auc_score

y_prob_rf = rf.predict_proba(X_test)[:, 1]

roc_auc_rf = roc_auc_score(y_test, y_prob_rf)

print("ROC-AUC RF:", roc_auc_rf)

ROC-AUC RF: 0.6554006968641115
```

El modelo Random Forest no aporta mejoras significativas respecto a la regresión logística, obteniendo métricas prácticamente idénticas tanto en capacidad de discriminación como en detección de pacientes en riesgo.

8.3.4. Gradient boosting

```
In [57]: from sklearn.ensemble import GradientBoostingClassifier

gb = GradientBoostingClassifier(random_state=42)

gb.fit(X_train, y_train)

y_pred_gb = gb.predict(X_test)

print(classification_report(y_test, y_pred_gb))

precision    recall  f1-score   support

0           0.70     0.50     0.59         105
1           0.53     0.72     0.61          82

accuracy          0.60         187
macro avg         0.61         187
weighted avg      0.62         187
```

```
In [58]: y_prob_gb = gb.predict_proba(X_test)[:, 1]

roc_auc_gb = roc_auc_score(y_test, y_prob_gb)

print("ROC-AUC GB:", roc_auc_gb)

ROC-AUC GB: 0.6554006968641115
```

Los modelos evaluados, incluyendo regresión logística, Random Forest y Gradient Boosting, presentan un rendimiento prácticamente idéntico en términos de capacidad predictiva, sin mejoras significativas al aumentar la complejidad del algoritmo. La ausencia de mejora al emplear modelos más complejos sugiere que la limitación principal del problema no reside en la capacidad del algoritmo, sino en la información contenida en los datos disponibles.

En este contexto, la regresión logística representa la mejor opción, al ofrecer un rendimiento equivalente a modelos más complejos, manteniendo al mismo tiempo una mayor interpretabilidad, especialmente relevante en entornos clínicos.

9. Aplicaciones del modelo de predicción

9.1. Predicción para un paciente nuevo (ejemplo)

```
In [59]: import pandas as pd

# nuevo paciente

nuevo_paciente = pd.DataFrame({
    "hta": [1],
    "colesterol": [0],
    "diabetes": [0],
    "n_grupos": [1]
})

# probabilidad de intensificación

prob = model.predict_proba(nuevo_paciente)[0][1]

print("Probabilidad de intensificación:", prob)

Probabilidad de intensificación: 0.573885707003161
```

Para medir el riesgo de intensificación, se usarán los siguientes parámetros:

<0.5 → bajo

0.5–0.8 → moderado

.>0.8 -> alto

Un paciente con hipertensión aislada presenta una probabilidad baja (~41%) de intensificar tratamiento a lo largo del periodo analizado.

9.2. Otros posibles escenarios

```
In [60]: escenarios = pd.DataFrame({
    "hta": [1, 1, 1],
    "colesterol": [0, 1, 1],
    "diabetes": [0, 0, 1],
```

```
        "n_grupos": [1, 2, 3]
    })

    escenarios["prob_intensificacion"] = model.predict_proba(escenarios)[: , 1]

    escenarios
```

Out[60]:

	hta	colesterol	diabetes	n_grupos	prob_intensificacion
0	1	0	0	1	0.573886
1	1	1	0	2	0.802129
2	1	1	1	3	0.915696

Se observa un incremento progresivo y no lineal en la probabilidad de intensificación terapéutica a medida que aumenta la carga metabólica inicial del paciente, pasando de un 40% en pacientes con hipertensión aislada a más del 90% en aquellos con tres factores concomitantes. La incorporación de diabetes al perfil del paciente supone un cambio cualitativo en la probabilidad de intensificación, actuando como principal driver de progresión dentro del síndrome metabólico.

Un paciente con HTA aislada puede ser monitorizado de forma estándar, mientras que un paciente con HTA, dislipemia y diabetes presenta una probabilidad muy elevada de intensificación, siendo candidato a seguimiento más estrecho.

9.3. Calculo de riesgo para pacietes del dataset

```
In [61]: df_model["probabilidad"] = model.predict_proba(
          df_model[["hta", "colesterol", "diabetes", "n_grupos"]]
        )[: , 1]

df_model.head()
```

Out[61]:

	hta	colesterol	diabetes	n_grupos	target	probabilidad
0	True	False	False	1	0	0.573886
1	True	False	False	1	0	0.573886
2	True	True	False	2	0	0.802129
3	True	False	False	1	0	0.573886
4	False	True	False	1	0	0.400732

9.4. Ordenar pacientes por riesgo y priorizar seguimiento

```
In [62]: df_model.sort_values(by="probabilidad", ascending=False).head(10)
```

Out[62]:

	hta	colesterol	diabetes	n_grupos	target	probabilidad
13	True	True	True	3	1	0.915696
141	True	True	True	3	0	0.915696
294	True	True	True	3	0	0.915696
98	True	True	True	3	0	0.915696
71	True	True	False	2	0	0.802129
155	True	True	False	2	0	0.802129
29	True	True	False	2	1	0.802129
2	True	True	False	2	0	0.802129
90	True	True	False	2	0	0.802129
145	True	True	False	2	1	0.802129

Target 1 hace referencia a pacientes que han intensificado el tratamiento; mientras que target 0 son pacientes que no lo han hecho. Se observa target = 0 en muchos casos de alto riesgo, es decir, que el modelo identifica pacientes con alta probabilidad de intensificación que aún no han experimentado cambios terapéuticos, lo que sugiere su utilidad como herramienta de predicción y no únicamente de análisis retrospectivo.

Los pacientes mostrados se han seleccionado como ejemplo por presentar las probabilidades más elevadas de intensificación terapéutica según el modelo, lo que los sitúa dentro del grupo de alto riesgo (>0.8). Esta selección permite ilustrar el potencial del modelo para identificar perfiles clínicos prioritarios, más allá de si han intensificado tratamiento previamente (target = 1) o no (target = 0).

Para facilitar la interpretación y aplicabilidad clínica, se ha establecido una estratificación del riesgo en tres niveles: bajo (<0.5), moderado (0.5–0.8) y alto (>0.8). Bajo este enfoque, los pacientes de alto riesgo constituyen el grupo de mayor interés, ya que presentan una elevada probabilidad de requerir intensificación terapéutica en el futuro, siendo candidatos a seguimiento más estrecho, revisión farmacológica y posibles intervenciones preventivas. Los pacientes de riesgo moderado podrían beneficiarse de monitorización periódica y educación sanitaria, mientras que aquellos de bajo riesgo seguirían un control estándar.

Un aspecto especialmente relevante es la presencia de pacientes con alta probabilidad de intensificación (por ejemplo, >0.9) que, sin embargo, no han intensificado tratamiento (target = 0), seguramente sea debido a que y se encontraban con ese tratamiento antes de la recogida de datos de 2025, y contarían como un falso positivo en cuanto al riesgo de progresión terapéutica; aún así, se trata de pacientes que siguen necesitando un seguimiento exhaustivo. Este hecho refuerza el carácter predictivo del modelo, ya que no se limita a describir lo ocurrido en el pasado, sino que permite anticipar qué pacientes tienen mayor riesgo de progresión terapéutica. En este sentido, el modelo actúa como una herramienta proactiva de estratificación, capaz de detectar pacientes en riesgo antes de que se produzcan cambios clínicos evidentes, lo que abre la puerta a intervenciones tempranas y a una gestión más eficiente del síndrome metabólico.

Este modelo permite pasar de una gestión reactiva a una estrategia proactiva, identificando de forma anticipada a los pacientes con mayor riesgo de intensificación terapéutica y

facilitando intervenciones dirigidas que optimicen el seguimiento clínico y los recursos asistenciales.

10. Evaluación de falsos positivos/negativos

10.1. Predicción de todo el dataset

```
In [63]: df_model["pred"] = model.predict(
         df_model[["hta", "colesterol", "diabetes", "n_grupos"]]
         )
```

10.2. Definir casos

```
In [64]: # falsos positivos
fp = df_model[(df_model["target"] == 0) & (df_model["pred"] == 1)]

# falsos negativos
fn = df_model[(df_model["target"] == 1) & (df_model["pred"] == 0)]
```

10.3. Comprobar cuántos hay

```
In [65]: print("Falsos positivos:", len(fp))
         print("Falsos negativos:", len(fn))
```

Falsos positivos: 146
Falsos negativos: 16

10.4. Analizar perfiles

```
In [66]: fp.groupby(["hta", "colesterol", "diabetes", "n_grupos"]).size()
```

Out[66]:

hta	colesterol	diabetes	n_grupos	
False	True	True	2	4
True	False	False	1	128
		True	2	2
	True	False	2	9
		True	3	3

dtype: int64

```
In [67]: fn.groupby(["hta", "colesterol", "diabetes", "n_grupos"]).size()
```

Out[67]:

hta	colesterol	diabetes	n_grupos	
False	False	True	1	8
	True	False	1	8

dtype: int64

```
In [68]: fp.sort_values(by="probabilidad", ascending=False).head()
```

Out[68]:

	hta	colesterol	diabetes	n_grupos	target	probabilidad	pred
294	True	True	True	3	0	0.915696	1
98	True	True	True	3	0	0.915696	1
141	True	True	True	3	0	0.915696	1
277	True	True	False	2	0	0.802129	1
10	True	True	False	2	0	0.802129	1

El presente estudio demuestra que es posible utilizar datos de dispensación farmacéutica para reconstruir la evolución terapéutica de los pacientes y desarrollar un modelo capaz de estimar la probabilidad de intensificación del tratamiento en el contexto del síndrome metabólico. A partir de variables indirectas, pero clínicamente relevantes, se ha conseguido identificar patrones de progresión y estratificar a los pacientes en función de su riesgo, evidenciando la utilidad de la ciencia de datos aplicada a entornos de práctica real.

Los resultados muestran que la intensificación terapéutica está estrechamente ligada a la acumulación de factores metabólicos, siendo la complejidad global del paciente el principal determinante del riesgo. Asimismo, el modelo presenta un comportamiento coherente desde el punto de vista clínico, con una mayor capacidad para identificar pacientes en fases avanzadas y ciertas limitaciones en etapas iniciales, lo que refuerza la importancia de la calidad y profundidad del dato disponible.

Más allá del análisis técnico, este trabajo pone de manifiesto el potencial de este tipo de modelos como herramienta aplicada en el ámbito sanitario. En particular, su implementación en entornos como la farmacia comunitaria permitiría evolucionar hacia un modelo de atención más proactivo, basado en la identificación temprana de pacientes con mayor riesgo de progresión y la personalización del seguimiento.

En este sentido, el modelo podría integrarse como un servicio de valor añadido, orientado a la monitorización de patrones relacionados con el síndrome metabólico (como tensión arterial, glucemia o perfil lipídico), facilitando un seguimiento continuo y accesible desde la farmacia. Además, la estratificación de pacientes permitiría identificar perfiles con mayor predisposición a intervenciones preventivas o complementarias, incluyendo recomendaciones en el ámbito de la parafarmacia (como suplementos dirigidos al control del apetito o del colesterol), siempre dentro de un enfoque responsable y basado en evidencia.

En conjunto, este proyecto no solo valida la viabilidad técnica del enfoque, sino que abre la puerta a su aplicación como herramienta de apoyo a la decisión clínica y como oportunidad de innovación en servicios asistenciales, especialmente en entornos donde la cercanía al paciente permite un seguimiento más continuo y personalizado.

El análisis de errores muestra que el modelo presenta un número elevado de falsos positivos (79) frente a un número reducido de falsos negativos (14), lo que indica una tendencia a sobredetectar pacientes en riesgo, priorizando la sensibilidad sobre la especificidad.

Los falsos positivos se concentran principalmente en pacientes con diabetes como única patología, lo que sugiere que el modelo identifica correctamente la diabetes como un factor de alto riesgo, pero tiende a sobreestimar su impacto en ausencia de otras variables clínicas. Parte de los falsos positivos identificados podrían corresponder a pacientes ya tratados de forma intensiva antes del periodo de estudio, lo que limita la capacidad del modelo para observar nuevas intensificaciones. Este fenómeno, conocido como censura temporal, implica que algunos casos clasificados como errores pueden reflejar en realidad limitaciones del dataset y no fallos del modelo.

En cambio, los falsos negativos aparecen en perfiles menos complejos, como pacientes con hipertensión aislada o combinaciones iniciales, evidenciando una limitación para captar progresiones tempranas o más sutiles. En conjunto, estos resultados reflejan que el modelo es eficaz identificando perfiles de alto riesgo, pero presenta limitaciones en la discriminación fina dentro de perfiles intermedios, probablemente debido a la ausencia de información clínica clave como control de la enfermedad o adherencia terapéutica.

Como lectura global, el modelo resulta útil como herramienta de priorización, aunque no sustituye el juicio clínico y debe interpretarse dentro del contexto de sus limitaciones.

11. Matriz de confusión

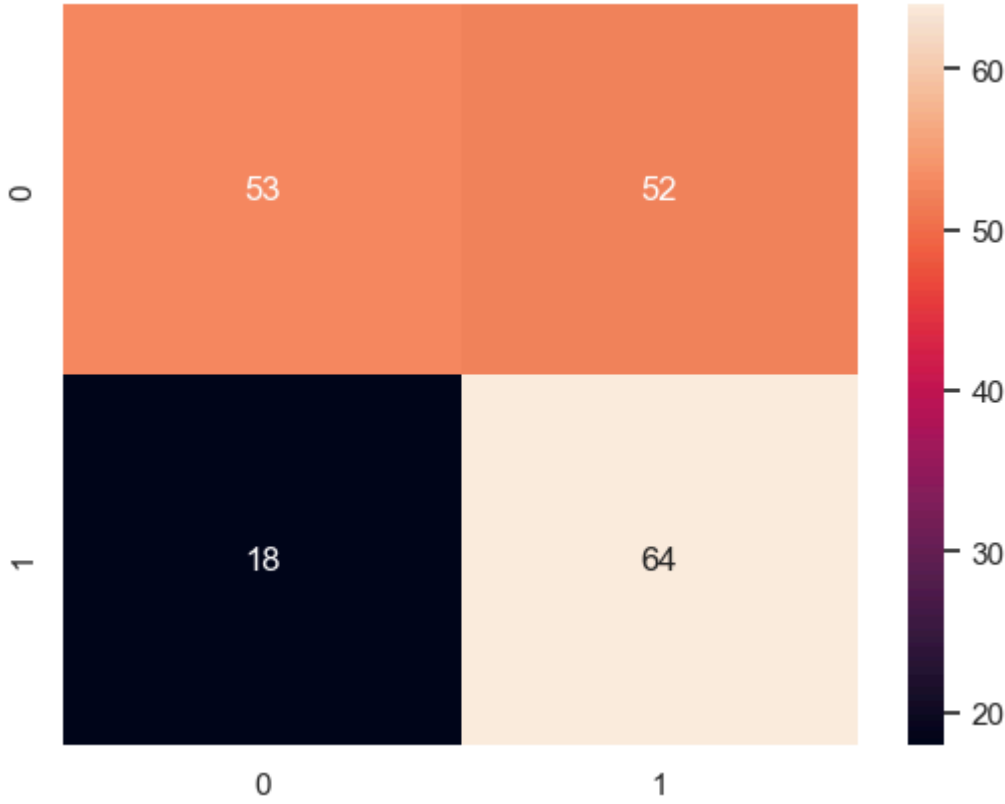
La matriz de confusión permite analizar en detalle el comportamiento del modelo, identificando no solo su capacidad de acierto, sino también el tipo de errores que comete. En este contexto, resulta especialmente relevante evaluar los falsos negativos, ya que implican pacientes que podrían requerir intensificación terapéutica y no son detectados por el modelo, lo que tiene mayor impacto clínico que los falsos positivos.

```
In [69]: from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt="d")
```

Out[69]: <Axes: >



En este caso, el modelo muestra una capacidad razonable para identificar pacientes que intensifican tratamiento (64 verdaderos positivos), aunque presenta un número relevante de falsos positivos (52), es decir, pacientes que el modelo clasifica como de alto riesgo pero que finalmente no intensifican.

Desde un punto de vista clínico, este comportamiento resulta aceptable e incluso deseable, ya que prioriza la detección de pacientes potencialmente en riesgo, reduciendo el número de falsos negativos (18), que representan los casos más críticos al implicar pacientes que intensifican tratamiento y no son detectados. En este sentido, el modelo se orienta hacia una estrategia conservadora, en la que se asume un mayor número de alertas a cambio de minimizar la pérdida de pacientes relevantes.

En conjunto, estos resultados refuerzan la utilidad del modelo como herramienta de apoyo a la estratificación de riesgo, especialmente en contextos donde la detección temprana resulta más relevante que la precisión estricta.

11.1. Análisis de censura temporal

En el análisis del modelo se observa la presencia de falsos positivos, es decir, pacientes clasificados como de alto riesgo que no han intensificado tratamiento durante el periodo estudiado. Sin embargo, esta aparente limitación puede explicarse en parte por un fenómeno de censura temporal, derivado del hecho de que los datos analizados se restringen al año 2025.

En este contexto, algunos pacientes pueden no presentar intensificación durante el periodo de estudio no porque no la requieran, sino porque ya se encuentran en un estadio terapéutico avanzado desde antes del inicio de la ventana temporal analizada. Como consecuencia, el modelo puede identificar correctamente perfiles de alto riesgo que no se traducen en eventos observables dentro del dataset, generando falsos positivos que en realidad reflejan limitaciones del dato más que errores del modelo.

Este fenómeno pone de manifiesto una limitación inherente a los datos de práctica real, donde la falta de información histórica impide capturar completamente la evolución del paciente. En este sentido, el modelo debe

interpretarse como una herramienta de estimación de riesgo potencial más que como un predictor exacto de eventos en un periodo cerrado, reforzando su utilidad en un enfoque proactivo de estratificación.

```
In [73]: # 1. identificar falsos positivos
# (predice intensificación = 1, pero en realidad no intensifica = 0)

fp = df_model[(df_model["pred"] == 1) & (df_model["target"] == 0)]

print("Número de falsos positivos:", len(fp))

# 2. analizar su perfil clínico
# agrupamos por variables clave

fp_perfiles = fp.groupby(["hta", "colesterol", "diabetes", "n_grupos"]).size()

print("\nDistribución de falsos positivos por perfil:")
print(fp_perfiles)

# 3. observar probabilidades altas dentro de los FP
# esto es clave para detectar posibles casos de censura

fp_alto_riesgo = fp.sort_values(by="probabilidad", ascending=False).head(10)

print("\nTop falsos positivos (mayor probabilidad):")
print(fp_alto_riesgo[["hta", "colesterol", "diabetes", "n_grupos", "probabilidad"]])

# 4. interpretación:
# si muchos FP tienen alta probabilidad y múltiples patologías,
# es posible que ya estuvieran en tratamiento intensivo antes de 2025
```

Número de falsos positivos: 146

Distribución de falsos positivos por perfil:

hta	colesterol	diabetes	n_grupos	
False	True	True	2	4
True	False	False	1	128
		True	2	2
	True	False	2	9
		True	3	3

dtype: int64

Top falsos positivos (mayor probabilidad):

	hta	colesterol	diabetes	n_grupos	probabilidad
294	True	True	True	3	0.915696
98	True	True	True	3	0.915696
141	True	True	True	3	0.915696
277	True	True	False	2	0.802129
10	True	True	False	2	0.802129
43	True	True	False	2	0.802129
155	True	True	False	2	0.802129
90	True	True	False	2	0.802129
71	True	True	False	2	0.802129
9	True	True	False	2	0.802129

El análisis de los falsos positivos muestra que una proporción significativa de estos pacientes presenta múltiples factores de riesgo, especialmente combinaciones que incluyen hipertensión, colesterol y diabetes (n_grupos ≥ 2). Además, muchos de estos casos presentan probabilidades elevadas de intensificación (>0.8), lo que indica que el modelo los identifica consistentemente como perfiles de alto riesgo.

Este patrón sugiere que estos falsos positivos no responden necesariamente a errores del modelo, sino que pueden estar condicionados por la ausencia de información previa al periodo analizado. En particular, es probable que algunos de estos pacientes ya se encuentren en un estadio terapéutico avanzado antes de 2025, lo que limita la observación de nuevas intensificaciones dentro del dataset. En este sentido, estos casos pueden interpretarse como evidencia de censura temporal más que como fallos del modelo.

```
In [74]: # falsos negativos
fn = df_model[(df_model["pred"] == 0) & (df_model["target"] == 1)]

print("Número de falsos negativos:", len(fn))

fn.groupby(["hta", "colesterol", "diabetes", "n_grupos"]).size()
```

Número de falsos negativos: 16

```
Out[74]:
```

hta	colesterol	diabetes	n_grupos	
False	False	True	1	8
	True	False	1	8

dtype: int64

El análisis de los falsos negativos muestra que estos casos se concentran principalmente en pacientes con una única patología (n_grupos = 1), tanto en perfiles asociados a diabetes como a hipertensión. Esto sugiere que el modelo presenta mayores dificultades para identificar procesos de intensificación en fases iniciales o menos complejas del síndrome metabólico, donde los patrones de progresión son menos evidentes.

A diferencia de los falsos positivos, estos casos no pueden explicarse por fenómenos de censura temporal, sino que reflejan limitaciones reales del modelo y de las variables disponibles. En particular, la ausencia de información clínica adicional (como valores analíticos o evolución de parámetros) puede dificultar la detección de cambios tempranos en pacientes aparentemente menos complejos.

Desde el punto de vista clínico, estos errores son los más relevantes, ya que implican pacientes que intensifican tratamiento y no son anticipados por el modelo, lo que pone de manifiesto áreas de mejora en la capacidad predictiva, especialmente en etapas iniciales de la enfermedad.

Los resultados sugieren la presencia de un fenómeno de censura temporal que afecta principalmente a los falsos positivos, donde pacientes identificados como de alto riesgo no presentan intensificación

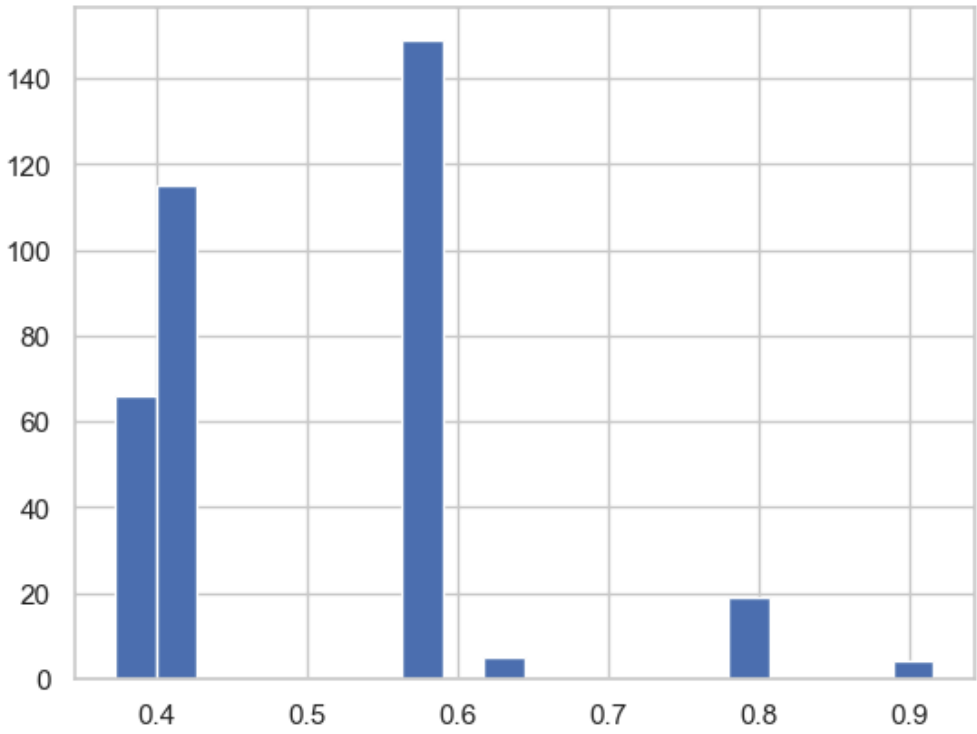
dentro del periodo analizado, posiblemente por encontrarse ya en un estadio terapéutico avanzado previo al inicio del estudio.

Sin embargo, este fenómeno no explica los falsos negativos, que se concentran en pacientes con menor complejidad y reflejan limitaciones reales del modelo para detectar fases iniciales de progresión.

12. Distribución de probabilidades

```
In [70]: plt.hist(df_model["probabilidad"], bins=20)
```

```
Out[70]: (array([ 66., 115.,   0.,   0.,   0.,   0.,   0., 149.,   0.,   5.,
   0.,   0.,   0.,   0.,  19.,   0.,   0.,   0.,   4.]),
 array([0.37314614, 0.40027366, 0.42740118, 0.45452869, 0.48165621,
 0.50878373, 0.53591124, 0.56303876, 0.59016628, 0.6172938 ,
 0.64442131, 0.67154883, 0.69867635, 0.72580386, 0.75293138,
 0.7800589 , 0.80718641, 0.83431393, 0.86144145, 0.88856897,
 0.91569648])),
<BarContainer object of 20 artists>)
```



La distribución de probabilidades estimadas por el modelo muestra una clara concentración de pacientes en rangos intermedios de riesgo, con una menor proporción de casos en valores extremos. Este patrón sugiere que el modelo discrimina de forma progresiva entre perfiles de bajo, medio y alto riesgo, evitando clasificaciones excesivamente binarias.

Asimismo, se observa la presencia de un grupo reducido de pacientes con probabilidades elevadas (>0.8), que constituyen el principal objetivo desde el punto de vista clínico, al representar aquellos con mayor probabilidad de intensificación terapéutica. La existencia de estos perfiles bien definidos refuerza la utilidad del modelo para la priorización de pacientes.

En conjunto, la distribución refleja un comportamiento coherente del modelo, capaz de generar una estratificación de riesgo continua y útil para la toma de decisiones, más allá de una simple clasificación dicotómica.

13. Coeficientes (feature importance)

```
In [72]: import pandas as pd
import matplotlib.pyplot as plt

# 1. extraer coeficientes del modelo
coeficientes = pd.Series(model.coef_[0], index=X_train.columns)

# 2. ordenar coeficientes
coeficientes_ordenados = coeficientes.sort_values()

# 3. mostrar por pantalla
print("Coeficientes ordenados:\n")
print(coeficientes_ordenados)

# 4. visualización
plt.figure(figsize=(8,5))
coeficientes_ordenados.plot(kind="barh")

plt.title("Importancia de variables (Regresión Logística)", fontstyle="italic")
plt.xlabel("Coeficiente")
plt.ylabel("Variables")

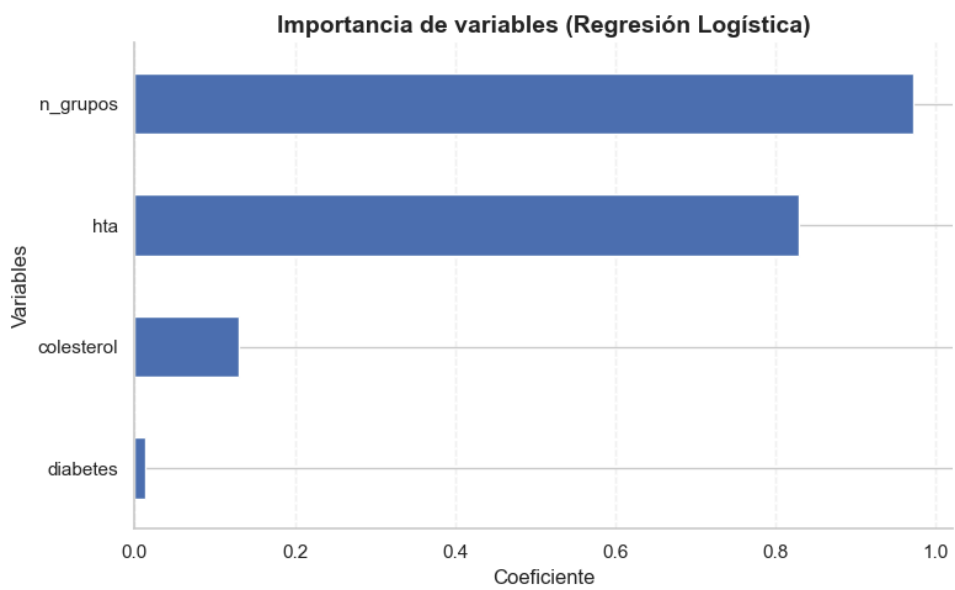
# grid limpio
plt.grid(axis="x", linestyle="--", alpha=0.2)

# quitar bordes innecesarios
for spine in ["top", "right"]:
    plt.gca().spines[spine].set_visible(False)

plt.tight_layout()
plt.show()
```

Coeficientes ordenados:

```
diabetes      0.013203
colesterol    0.129530
hta           0.829669
n_grupos      0.972403
dtype: float64
```



El análisis de los coeficientes del modelo muestra que la variable con mayor impacto en la predicción de intensificación terapéutica es el número de grupos terapéuticos (n_grupos), seguida de la presencia de hipertensión (hta). Este resultado indica que la complejidad global del paciente y la coexistencia de múltiples patologías son los principales determinantes del riesgo de intensificación.

Por el contrario, las variables individuales de colesterol y diabetes presentan una menor contribución relativa al modelo, lo que sugiere que su impacto es más relevante en combinación con otros factores que de forma aislada. Este patrón es coherente con la fisiopatología del síndrome metabólico, donde el riesgo no depende de una única condición, sino de la acumulación progresiva de alteraciones metabólicas.

En conjunto, estos resultados refuerzan la interpretabilidad del modelo y validan su coherencia clínica, al identificar como principales predictores aquellos elementos asociados a una mayor carga metabólica y complejidad del paciente.

Limitaciones del modelo

El presente estudio se basa en datos de dispensación farmacéutica, lo que aporta un enfoque de práctica real, pero también introduce una serie de limitaciones que deben considerarse en la interpretación de los resultados.

En primer lugar, las variables utilizadas actúan como proxies clínicos, ya que la presencia de patologías como hipertensión, dislipemia o diabetes se infiere a partir del tratamiento farmacológico y no de parámetros clínicos directos. Esto implica que el modelo no captura la totalidad de la información médica del paciente, sino una aproximación basada en su tratamiento.

Asimismo, existe una limitación temporal (censura temporal) derivada de la restricción del análisis al año 2025. Algunos pacientes pueden no mostrar intensificación durante este periodo no porque no la requieran, sino porque ya se encuentran en fases avanzadas de tratamiento previamente, lo que puede generar falsos positivos que reflejan limitaciones del dato más que errores del modelo.

Por otro lado, los datos proceden de un entorno de dispensación, lo que introduce ruido estructural, como la posible agregación de consumos de distintos individuos bajo un mismo identificador (por ejemplo, compras para familiares). Este fenómeno puede afectar a la reconstrucción de la evolución terapéutica y a la precisión del modelo.

Además, la ausencia de variables clínicas adicionales (como valores analíticos, adherencia al tratamiento o información diagnóstica) limita la capacidad del modelo para detectar fases iniciales de progresión, lo que se refleja en la presencia de falsos negativos en pacientes con menor complejidad.

Por último, el estudio se basa en datos de una única farmacia, lo que condiciona la generalización de los resultados, ya que los patrones observados pueden no ser completamente extrapolables a otros entornos o poblaciones.

Conclusiones del estudio y discusión de resultados

El presente estudio demuestra que es posible utilizar datos de dispensación farmacéutica para reconstruir la evolución terapéutica de los pacientes y desarrollar un modelo capaz de estimar la probabilidad de intensificación del tratamiento en el contexto del síndrome metabólico. A partir de variables indirectas, pero clínicamente relevantes, se ha conseguido identificar patrones de progresión y estratificar a los pacientes en función de su riesgo, evidenciando la utilidad de la ciencia de datos aplicada a entornos de práctica real.

Los resultados muestran que la intensificación terapéutica está estrechamente ligada a la acumulación de factores metabólicos, siendo la complejidad global del paciente el principal determinante del riesgo. Asimismo, el modelo presenta un comportamiento coherente desde el punto de vista clínico, con una mayor capacidad para identificar pacientes en fases avanzadas y ciertas limitaciones en etapas iniciales, lo que refuerza la importancia de la calidad y profundidad del dato disponible.

Más allá del análisis técnico, este trabajo pone de manifiesto el potencial de este tipo de modelos como herramienta aplicada en el ámbito sanitario. En particular, su implementación en entornos como la farmacia comunitaria permitiría evolucionar hacia un modelo de atención más proactivo, basado en la identificación temprana de pacientes con mayor riesgo de progresión y la personalización del seguimiento.

En este sentido, el modelo podría integrarse como un servicio de valor añadido, orientado a la monitorización de patrones relacionados con el síndrome metabólico (como tensión arterial, glucemia o perfil lipídico), facilitando un seguimiento continuo y accesible desde la farmacia. Además, la estratificación de pacientes permitiría identificar perfiles con mayor predisposición a intervenciones preventivas o complementarias, incluyendo recomendaciones en el ámbito de la parafarmacia (como suplementos dirigidos al control del apetito o del colesterol), siempre dentro de un enfoque responsable y basado en evidencia.

En conjunto, este proyecto no solo valida la viabilidad técnica del enfoque, sino que abre la puerta a su aplicación como herramienta de apoyo a la decisión clínica y como oportunidad de innovación en servicios asistenciales, especialmente en entornos donde la cercanía al paciente permite un seguimiento más continuo y personalizado.